

ORGANIZATIONAL KNOWLEDGE GAP INTELLIGENCE PLATFORM

SOFTWARE PROJECT DOCUMENTATION

Organization: Infosys

Project Duration: 4 Weeks

Individual Contribution: Backend Development

Submitted by N.SRIRANJANI

ABSTRACT

1. Abstract

The Organizational Knowledge Gap Intelligence Platform (OKGIP) is a full-stack enterprise application built to help organizations identify, quantify, and close skill gaps between what an employee currently knows and what their role requires. The system was developed over an 8-week engineering effort at Infosys and is organized into three delivery milestones: (1) authentication, employee profiles, and the competency framework; (2) a gap-analysis engine and a training-recommendation engine; and (3) mentorship/knowledge-sharing, learning-progress tracking, skills assessments, analytics dashboards, and an in-app notification system.

The verified technology stack is a Java 17 / Spring Boot 3.3.4 backend exposing a stateless REST API, secured with Spring Security and JSON Web Tokens (JWT) issued and validated through a custom filter chain, persisting data through Spring Data JPA/Hibernate to either an in-memory H2 database (default, development profile) or PostgreSQL (production profile, provisioned via Docker Compose). The frontend is a React 18 single-page application built with Vite, styled with Tailwind CSS, routed with React Router, and communicating with the backend through an Axios client that attaches the JWT to every request.

Functionally, the platform lets an employee record self-assessed skill proficiency, compares it against a configurable role/department competency framework, and computes a weighted gap and severity score per skill. These gaps drive a transparent, rule-based training-recommendation engine (internal academy and external providers), a generated multi-step learning path, and a mentor-matching suggestion list based on skill complementarity. Employees can enroll in training, update progress, and take multiple-choice skill assessments that are graded server-side and feed back into the employee's proficiency record, automatically refreshing their gap analysis. Manager and organization-wide analytics dashboards aggregate this data into department readiness scores, gap heatmaps, and training-effectiveness metrics, rendered with Recharts. An in-app notification inbox surfaces milestone, mentorship, and assessment events to each user.

This document distinguishes clearly, section by section, between functionality that is actually implemented in the supplied source code, functionality that is partially implemented, and technologies referenced only as future or proposed direction (for example, Redis, Kafka, Elasticsearch, OpenAI/LLM-backed recommendations, and containerized deployment of the application tiers themselves are not present in the codebase and are discussed only in Section 11, Future Enhancements). No test suite, CI/CD pipeline, or performance benchmark exists in the provided source, and this is stated plainly rather than assumed. Numerical outcomes not present in the codebase or supplied materials are not fabricated; where a metric cannot be verified, this document says so explicitly.

My individual contribution to this project was backend development: the Spring Boot application layer, comprising nine REST controllers, eight service classes containing the domain logic (in particular the gap-analysis, recommendation, and assessment-grading algorithms), thirteen Spring Data JPA repositories, thirteen JPA entities, the JWT-based Spring Security configuration, and the demonstration data seeder used to populate a working environment for the frontend team. Sections 6 and 7 of this document give this backend work additional technical depth, consistent with my role on the team.

PROJECT OVERVIEW & OBJECTIVES

2.1 Introduction

Large organizations struggle to answer a deceptively simple question: which employees are missing which skills, relative to what their job actually requires? Skills data is usually scattered across spreadsheets, LMS exports, and manager memory, and by the time a gap is noticed it has often already produced a delivery risk. The Organizational Knowledge Gap Intelligence Platform centralizes this problem into a single application: employees and managers get a live, quantified view of skill gaps, and the system turns those gaps directly into recommended training and mentorship actions.

Concretely, the implemented platform helps an organization to:

- Record each employee's self-declared (and, through assessments, system-derived) skill proficiency across an eleven-skill catalog seeded in the demonstration data.
- Compare current proficiency against a required-proficiency competency framework defined per job role and department.
- Quantify the resulting knowledge deficiency as a per-skill gap value, a severity band, and a numeric severity score, and roll these up into an overall readiness percentage.
- Recommend specific internal and external training programs that close the highest-priority gaps, and sequence them into a learning path.
- Track enrollment and completion of that training, and automatically re-evaluate the employee's skill level on completion.
- Improve organization-wide visibility through manager and organization dashboards showing department readiness, gap heatmaps, and training-provider effectiveness.

2.2 Problem Statement

Business problem: organizations cannot reliably answer "who is ready for this role, and what do they need to get there?" without a system that continuously compares actual skill levels to role requirements. Technical problem: this requires (a) a durable model of required competencies per role, (b) a durable, queryable model of each employee's current proficiency, (c) a deterministic and explainable way to compute the delta between the two, and (d) a way to turn that delta into concrete next actions (training, mentorship, assessment) rather than a static report.

2.3 Existing System

The requirements framing for this project (reflected in the README's description of prior practice) describes conventional approaches as relying on periodic, manually compiled skills matrices, disconnected LMS completion records, and manager judgement calls that are not tied to a quantitative required-vs-current comparison. These approaches do not recompute automatically when an employee completes training or passes an assessment, and they do not generate ranked training recommendations. This characterization is a design rationale carried in the project's own documentation rather than a claim independently verified against a specific legacy system, and is presented here as such.

2.4 Proposed System

The proposed and implemented system is a single full-stack application in which the competency framework, employee skill inventory, and gap computation live in one relational schema and are served through one authenticated REST API. Gap computation (Section 5.4) runs on demand from live data rather than a periodic batch export, so it reflects the current state immediately after a skill update, a completed training enrollment, or a graded assessment.

2.5 Core Objectives

Objective	Status
Employee profile management	Implemented — registration, login, profile fields (department, job role) via AuthController/ProfileController.
Skill inventory	Implemented — EmployeeSkill entity, GET/PUT /api/me/skills.
Competency assessment	Implemented (framework) — RoleCompetency entity and /api/competencies; quiz-based proficiency assessment also implemented (Section 5.8).
Knowledge-gap analysis	Implemented — GapAnalysisService (Section 5.4).
Training recommendations	Implemented — RecommendationService, rule-based scoring (Section 5.5).
Mentorship	Implemented — MentorshipService matching and request workflow (Section 5.6).
Learning tracking	Implemented — LearningService enrollment/progress/completion (Section 5.7).
Assessments	Implemented — AssessmentService, multiple-choice quizzes with server-side grading (Section 5.8).
Notifications	Implemented — in-app NotificationService/NotificationController; no email/SMS/push channel implemented.
Analytics	Implemented — AnalyticsService (manager and organization dashboards), Recharts visualizations.
Reporting (PDF/Excel export)	Not Verified in the provided source code — no export/report-generation code found.

2.6 Project Scope

In scope and implemented: authentication and registration; skill inventory management; a static, seed-configured competency framework; individual and aggregate gap analysis; a rule-based recommendation engine and generated learning path; training enrollment and progress tracking with skill uplift on completion; mentorship suggestion and request workflow; knowledge-sharing sessions and articles (read/register only — no create/edit UI was found for articles or sessions beyond the seeded data); quiz-based assessments with automated grading; manager and organization analytics dashboards; and an in-app notification inbox.

Out of scope for this build (see Section 11 for the future-enhancement framing): automated or AI-generated recommendations (the recommendation engine is deterministic, not LLM-backed, though

the code comments explicitly note this as a possible future extension point); email, SMS, or push notification delivery; PDF/Excel report export; containerized deployment of the application tiers themselves; and any automated test suite.

2.7 Key Outcomes

The 8-week effort produced a working, demonstrable full-stack application: 9 REST controllers, 8 service classes, 13 JPA repositories, 13 JPA entities, and a 10-route React frontend, backed by a seed dataset of 8 demo users across 5 departments/roles, 11 skills, 24 role-competency requirements, 13 training programs, 6 assessments comprising 45 multiple-choice questions, 3 knowledge-sharing sessions, and 3 knowledge articles (all counts verified directly against DataSeeder.java). Exact performance, defect, or velocity metrics for the 8-week period were not present in the supplied source or documentation and are therefore not stated as numbers in this report.

SYSTEM REQUIREMENTS & TECHNOLOGY STACK

3.1 Frontend Technology

Technology	Implementation Status
JavaScript (ES modules)	Implemented — all frontend source is .jsx/.js using ES module imports.
React 18.3.1	Implemented — verified in frontend/package.json; functional components with hooks throughout.
React Router 6.26.2	Implemented — verified in App.jsx (10 routes, protected-route wrapper).
Axios 1.7.7	Implemented — verified in src/api/client.js, a single configured instance with request/response interceptors.
Tailwind CSS 3.4.13	Implemented — verified in tailwind.config.js, postcss.config.js, and utility classes used across all pages.
Chart.js	Not Verified in the provided source code — not present in package.json or imports.
Recharts 2.12.7	Implemented — verified in package.json and used in pages/Analytics.jsx (BarChart, PieChart).
D3.js	Not Verified in the provided source code — not present in package.json or imports.
Context API	Implemented — AuthContext.jsx provides authentication state (user, login, register, logout) app-wide; no other React Context providers were found.

3.2 Backend Technology

Technology	Implementation Status
Java 17	Implemented — <java.version>17</java.version> in pom.xml.
Spring Boot 3.3.4	Implemented — spring-boot-starter-parent version 3.3.4.
Spring Security	Implemented — SecurityConfig.java: stateless filter chain, custom JWT filter, BCrypt password encoding, CORS configuration.
Spring Data JPA	Implemented — spring-boot-starter-data-jpa; 13 JpaRepository interfaces.
Hibernate	Implemented — the JPA provider bundled with Spring Data JPA; ddl-auto: update is configured.
Maven	Implemented — the project is a standard single-module Maven build (pom.xml, no wrapper scripts present in the archive).
Spring WebSocket	Not Verified in the provided source code — no WebSocket dependency, configuration, or endpoint found.
REST APIs	Implemented — 9 @RestController classes exposing 35 JSON endpoints under /api/**.
JWT	Implemented — io.jsonwebtoken (jjwt-api/impl/jackson) 0.11.5; JwtService issues and validates HS256-signed tokens; JwtAuthFilter authenticates each request.

Technology	Implementation Status
OAuth2	Not Implemented — no spring-security-oauth2 dependency and no OAuth2 client/resource-server configuration. The word "OAuth2" appears only inside seeded skill-description and quiz text, not in security configuration.

3.3 Database & Storage

Technology	Implementation Status
H2 (in-memory)	Implemented — default active profile; jdbc:h2:mem:kgip with PostgreSQL compatibility mode; H2 console enabled at /h2-console.
PostgreSQL	Implemented as an alternate Spring profile — application.yml defines a postgres profile pointing at jdbc:postgresql://localhost:5432/kgip, and docker-compose.yml provisions a Postgres 16 container for it. Not the default profile.
Redis	Not Verified in the provided source code — no Redis dependency or configuration. Referenced only as a distractor answer inside a seeded quiz question ("Why is a server session not needed with JWT — because the token carries identity, not because sessions are stored in Redis").
Elasticsearch	Not Verified in the provided source code.

3.4 Messaging & External Integrations

Technology	Implementation Status
Apache Kafka	Not Implemented — no dependency, configuration, or producer/consumer code found.
RabbitMQ	Not Implemented — no dependency or configuration found.
OpenAI API / LLM	Not Implemented — RecommendationService carries a code comment stating its transparent scoring function is "deliberately transparent so it can be swapped for an LLM-backed ranker (OpenAI) without changing the API contract," which documents design intent, not an implementation. No API client, key configuration, or call to an LLM endpoint exists.
Twilio	Not Implemented — no dependency or configuration found.
Firebase Cloud Messaging	Not Implemented — no dependency or configuration found.
JavaMailSender	Not Implemented — no mail dependency or configuration found. Notifications are persisted rows served over REST (Section 5.9), not emails.

3.5 Development Environment

Item	Detail
Java version	17 (pom.xml <java.version>)

Item	Detail
Node.js version	Not pinned in the archive — no .nvmrc or engines field in package.json; Vite 5.4.x and the dependency set are compatible with Node 18+.
npm	Used for frontend dependency management (package.json / npm install, npm run dev per README).
Maven	Used for backend build/run (mvn spring-boot:run per README); no Maven Wrapper (mvnw) files were present in the archive despite the README referencing ./mvnw.
IDE	Not Verified in the provided source code — README recommends VS Code with the Java/Spring Boot/ESLint/Tailwind extension packs, but no IDE project files (.idea, .vscode content) were included.
Operating system	Not applicable / not verified — no OS-specific scripts or paths were found.
Build tools	Maven (backend), Vite 5.4.8 (frontend bundler/dev server).

3.6 Technology Stack Summary Table

Technology	Layer	Purpose	Implementation Status
Java 17	Backend runtime	Language runtime for the Spring Boot application	Implemented
Spring Boot 3.3.4	Backend framework	Application bootstrap, auto-configuration, embedded server	Implemented
Spring Security	Backend / security	Authentication and request-level authorization	Implemented
Spring Data JPA / Hibernate	Backend / persistence	ORM and repository abstraction over the relational schema	Implemented
JWT 0.11.5	Backend / security	JWT generation and validation	Implemented
H2 Database	Data layer (dev)	Default in-memory relational store, seeded on boot	Implemented
PostgreSQL	Data layer (prod profile)	Production-grade relational store via the postgres Spring profile	Implemented (alt. profile)
Maven	Build	Backend dependency management and build lifecycle	Implemented
React 18.3.1	Frontend framework	Component-based single-page application UI	Implemented
Vite 5.4.8	Frontend build	Dev server and production bundler	Implemented
React Router 6.26.2	Frontend routing	Client-side route matching and protected routes	Implemented
Axios 1.7.7	Frontend / HTTP	REST client with JWT-attaching interceptor	Implemented
Tailwind CSS 3.4.13	Frontend styling	Utility-first styling of all pages/components	Implemented

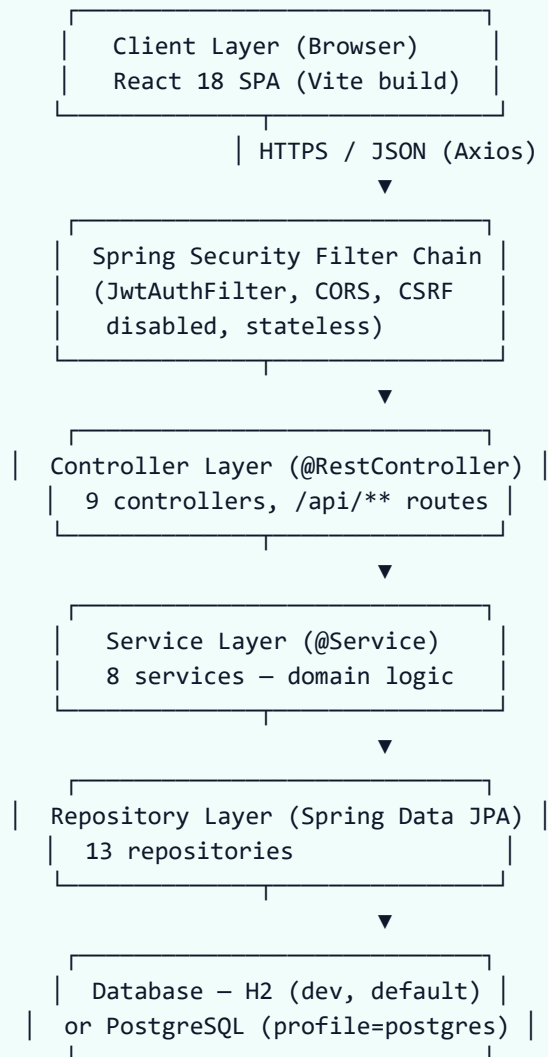
Technology	Layer	Purpose	Implementation Status
Recharts 2.12.7	Frontend / data viz	Bar and pie charts on the Analytics dashboard	Implemented
Docker Compose	Infrastructure (partial)	Provisions only a standalone PostgreSQL 16 container	Partially Implemented
JUnit / Mockito / RTL	Testing	Declared as Maven test-scope dependencies only	Not Verified

SYSTEM ARCHITECTURE & TECHNICAL DESIGN

4.1 Overall System Architecture Diagram

The implemented architecture is a classic three-tier web application: a React single-page client, a stateless Spring Boot REST API, and a relational database. There is no API gateway, no service mesh, no message broker, and no microservice decomposition in the supplied source — the backend is a single deployable Spring Boot artifact (a monolith), which is appropriate for the project's scope.

Figure 4.1 — Overall System Architecture (Implemented)



4.2 Backend Layered Architecture

The backend follows a conventional four-layer structure, using the actual package names under `com.okgip`:

Package	Responsibility (verified)
<code>com.okgip.web</code>	Controller layer — 9 <code>@RestController</code> classes translating HTTP requests to service calls.
<code>com.okgip.service</code>	Service layer — 8 <code>@Service</code> classes holding all business logic (gap analysis, recommendations, grading, analytics, etc.).
<code>com.okgip.repo</code>	Repository layer — 13 interfaces extending <code>JpaRepository<T, Long></code> .
<code>com.okgip.model</code>	Entity/model layer — 13 <code>@Entity</code> classes and 4 enums (<code>Role</code> , <code>Proficiency</code> , <code>AssessmentType</code> , <code>TrainingStatus</code>).
<code>com.okgip.dto</code>	Data-transfer layer — a single <code>Dtos.java</code> class containing ~30 nested Java records (one namespace, not one file per DTO).
<code>com.okgip.config</code>	Cross-cutting configuration — <code>SecurityConfig</code> , <code>JwtService</code> , <code>JwtAuthFilter</code> , <code>CurrentUser</code> .
<code>com.okgip.bootstrap</code>	Startup — <code>DataSeeder</code> , a <code>CommandLineRunner</code> that populates demo data once on first boot.

There is no separate exception-handling package: services throw Spring's built-in `ResponseStatusException` directly (e.g., 401 for invalid credentials, 404 for a missing entity, 403 for acting on another user's resource) and no `@ControllerAdvice` / global exception handler class was found in the archive. This is documented factually in Section 6.13 rather than assumed to exist.

4.3 Entity-Relationship Diagram

The schema below is derived directly from the `@Entity` classes and their `@ManyToOne` mappings in `com.okgip.model`. Hibernate manages the schema with `ddl-auto: update`, so no committed SQL DDL or migration files (e.g., Flyway/Liquibase) exist in the archive.

Table (entity)	Primary Key	Foreign Keys	Other Columns
users	id (PK)	—	email, password (BCrypt hash), fullName, role (enum), department, jobRole, avatarInitials, active
employee_skill	id (PK)	user_id → users, skill_id → skill	level (enum Proficiency), source (SELF/PEER/MANAGER/ASSESSMENT/TRAINING)
skill	id (PK)	—	name (unique), category, description
role_competency	id (PK)	skill_id → skill	jobRole, department, requiredLevel (enum), critical (boolean)
assessment	id (PK)	skill_id → skill	title, description, type (enum), timeLimitMinutes
question	id (PK)	assessment_id → assessment	text, optionA-D, correctOption, explanation, weight

Table (entity)	Primary Key	Foreign Keys	Other Columns
assessment_attempt	id (PK)	user_id → users, assessment_id → assessment	score, total, percent, derivedLevel (enum), submittedAt
training_program	id (PK)	skill_id → skill	title, provider, url, targetLevel (enum), durationHours, internal, description
enrollment	id (PK)	user_id → users, program_id → training_program	status (enum), progressPercent, enrolledAt, completedAt, dueAt
mentorship_match	id (PK)	mentor_id → users, mentee_id → users, skill_id → skill	matchScore, status (string), createdAt
sharing_session	id (PK)	host_id → users, skill_id → skill	title, description, scheduledAt, seats, registered
knowledge_article	id (PK)	author_id → users, skill_id → skill	title, content, createdAt
notification	id (PK)	user_id → users	title, message, type (string), readFlag, createdAt

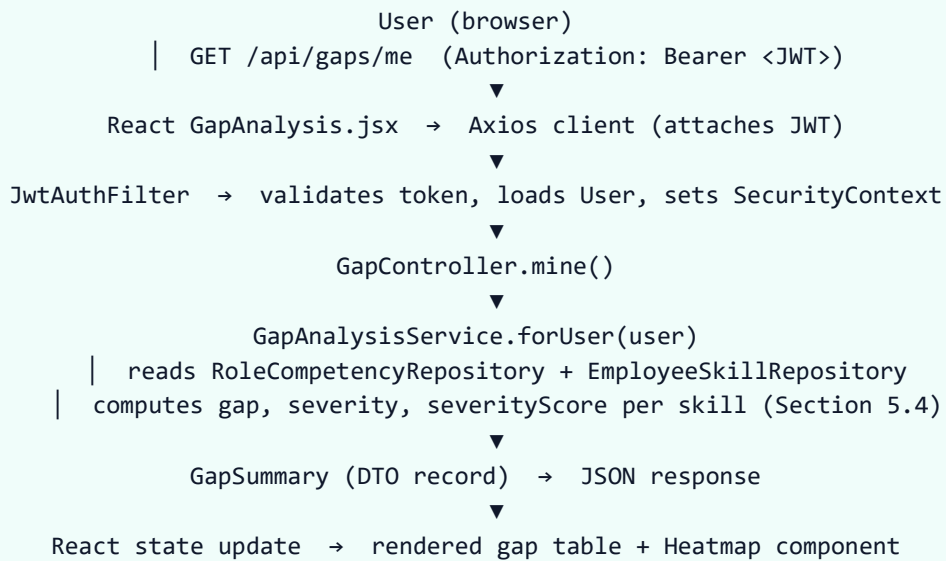
Cardinality (verified from @ManyToOne annotations; every mapping below is many-to-one from the child toward users/skill, i.e. the inverse is one-to-many):

- One user has many employee_skill rows (1:N) — a user's skill inventory.
- One skill is referenced by many employee_skill, role_competency, assessment, training_program, sharing_session, knowledge_article, and mentorship_match rows (1:N in each case).
- One user has many enrollment rows (1:N); one training_program has many enrollment rows (1:N).
- One assessment has many question rows (1:N) and many assessment_attempt rows (1:N).
- One user can be the mentor on many mentorship_match rows and the mentee on many others (two separate 1:N relationships from users to mentorship_match).
- One user has many notification rows (1:N).

No explicit @OneToMany collections were declared on the User or Skill entities — all navigation in the code goes through repository query methods (e.g., findByIdUserId, findByIdSkillId) rather than JPA-managed collections, and no @JoinTable / many-to-many mapping exists anywhere in the model.

4.4 Control Flow / Component Interaction Diagram

Representative flow — an employee opening the Gap Analysis page (verified against GapController → GapAnalysisService):

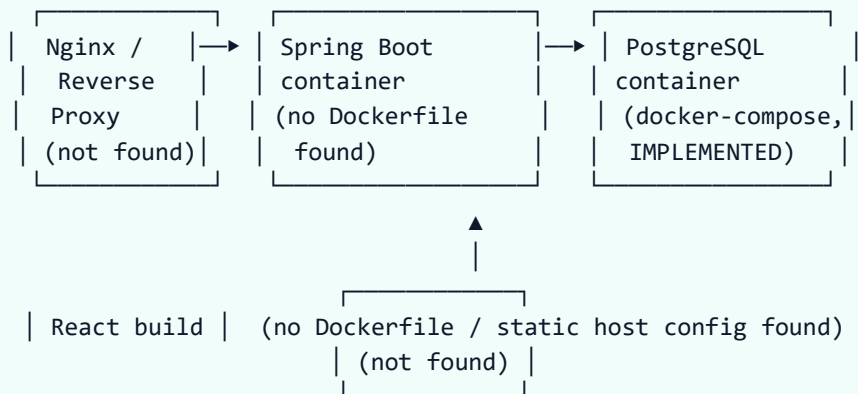
Figure 4.2 — Gap Analysis Request Flow (Implemented)

4.5 Data Flow Diagram

User input (skill self-ratings, assessment answers, enrollment actions) is submitted from React pages through the shared Axios instance to the corresponding controller. Controllers perform no business logic themselves — they extract the authenticated User via the `CurrentUser` helper and delegate immediately to a service. Services read/write through Spring Data JPA repositories, and state-changing operations (enrollment completion, assessment submission, mentorship requests, session registration) additionally call `NotificationService.push(...)` to persist an in-app notification row for the affected user(s). There is no asynchronous or background job processing (no `@Async`, `@Scheduled`, or queue-consumer code was found) — every write happens synchronously within the HTTP request thread that triggered it.

4.6 Deployment Architecture

The only deployment artifact present in the archive is `docker-compose.yml`, and it provisions a single service: a PostgreSQL 16 container (kgip database, default postgres/postgres credentials, a named volume for data persistence). There is no Dockerfile for the Spring Boot backend, no Dockerfile for the React frontend, and no reverse proxy (e.g., Nginx) configuration in the archive. Running the application therefore currently means: start the Postgres container (or rely on the default embedded H2 database), run the backend directly with Maven, and run the frontend directly with the Vite dev server — as described in `README.md`.

Figure 4.3 — Deployment (PROPOSED / FUTURE ARCHITECTURE)

MODULE-WISE DETAILED IMPLEMENTATION

This section documents all 12 modules defined by the project requirements. Each module states its purpose, functional requirements, user workflow, frontend/backend implementation, REST APIs, database entities, business logic, validation, error handling, a screenshot placeholder, and its verified implementation status.

5.1 User Authentication & Role-Based Access Control

Purpose

Authenticate users and issue a token that identifies them, their role, and their identity claims on every subsequent request.

Functional Requirements

- Register a new account with email, password, name, department, and job role.
- Log in with email and password and receive a signed token.
- Retrieve the current authenticated user's profile.

User Workflow

A user submits the Register or Login form (frontend pages Register.jsx / Login.jsx) → AuthContext calls the API → on success the JWT is stored in localStorage (kgip_token) and the user object in kgip_user → subsequent Axios requests attach Authorization: Bearer <token> via a request interceptor.

Frontend Interaction

Login.jsx and Register.jsx call AuthContext.login()/register(); AuthContext persists the token/user pair to localStorage and exposes them through React Context; App.jsx's Protected wrapper redirects to /login when no user is present.

Backend Implementation

AuthController delegates to AuthService, which looks up the user by email, verifies the password with BCryptPasswordEncoder.matches(...), and calls JwtService.generateToken(email, claims) to mint a token carrying the user's role and name as custom claims. Registration additionally checks UserRepository.existsByEmail(...) and rejects duplicates with HTTP 409.

REST APIs

Method	Endpoint	Auth Required
POST	/api/auth/login	No
POST	/api/auth/register	No
GET	/api/auth/me	Yes

Database Entities/Tables

users (email unique, password stores a BCrypt hash, role enum EMPLOYEE/MANAGER/HR/DEPARTMENT_HEAD/LD_ADMIN/ADMIN).

Business Logic

Password hashing via BCrypt at registration; on registration a role defaulting to EMPLOYEE if none supplied; avatar initials derived from the full name (first + last initial).

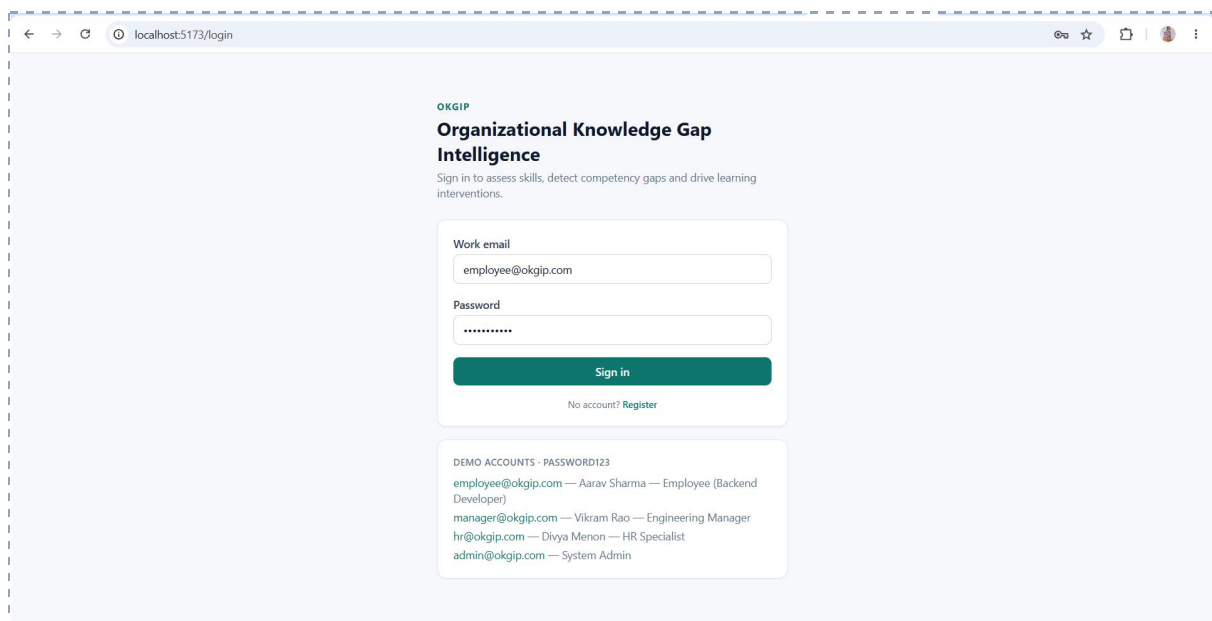
Validation

No jakarta.validation annotations (@NotBlank, @Email, etc.) were found on LoginRequest/RegisterRequest despite spring-boot-starter-validation being a declared Maven dependency; input validation is effectively limited to the duplicate-email check and whatever the database column constraints enforce. This is stated as an observation, not a defect assumption — see Section 9.8.

Error Handling

Invalid credentials → HTTP 401 via ResponseStatusException; duplicate registration email → HTTP 409.

Screenshot Placeholder



Implementation Status: **Implemented**

Role-Based Access Control — implementation depth

The Role enum defines six values (EMPLOYEE, MANAGER, HR, DEPARTMENT_HEAD, LD_ADMIN, ADMIN) and every issued JWT carries the user's role as a claim, and JwtAuthFilter grants a matching ROLE_<name> Spring Security authority. However, SecurityConfig.filterChain(...) only applies an explicit hasAnyRole(...) restriction to one route group — GET /api/competencies/**, which is restricted to HR, LD_ADMIN, ADMIN, DEPARTMENT_HEAD, MANAGER, EMPLOYEE (i.e., every defined role, which is functionally equivalent to "any authenticated user"). Every other authenticated route matches .anyRequest().authenticated() with no per-role restriction at the security-filter level. Where the code does restrict access by ownership (e.g., a user may only update or delete their own enrollment), that check is implemented manually inside the service layer (comparing e.getUser().getId() to the current user's id) rather than through Spring Security method

security such as `@PreAuthorize`. This distinction is documented plainly here and revisited in Section 9.8 and Section 10.

5.2 Employee Profile & Skill Inventory

Purpose

Let an employee record and view their proficiency across the organization's skill catalog.

Functional Requirements

- List all catalog skills.
- List the current user's own skill inventory with level and source.
- Create or update a proficiency rating for a skill.

User Workflow

`SkillProfile.jsx` loads `/api/skills` and `/api/me/skills` in parallel, renders each skill with a level selector, and PUTs an update to `/api/me/skills` when the employee changes a rating.

Frontend Interaction

Two parallel GET calls via `Promise.all` in `SkillProfile.jsx`; a PUT call on save.

Backend Implementation

`ProfileController` reads/writes directly through `SkillRepository` and `EmployeeSkillRepository` (no dedicated `SkillService` class exists — this controller performs simple CRUD calls directly against repositories, unlike the other domain areas which route through a service class).

REST APIs

Method	Endpoint	Auth Required
GET	<code>/api/skills</code>	Yes
GET	<code>/api/me/skills</code>	Yes
PUT	<code>/api/me/skills</code>	Yes

Database Entities/Tables

`skill`, `employee_skill` (composite lookup by `user_id` + `skill_id` via `findByUserIdAndSkillId`).

Business Logic

Upsert semantics: if an `EmployeeSkill` row exists for (user, skill) it is updated in place; otherwise a new row is created. The source field defaults to "SELF" when the client does not supply one.

Validation

None beyond the existence check on the skill id (`skills.findById(...).orElseThrow()`); no bounds checking is applied to the level enum beyond normal JSON deserialization.

Error Handling

An unknown `skillId` throws `NoSuchElementException` from `orElseThrow()` with no explicit HTTP-status mapping, which Spring's default handling will surface as a generic HTTP 500 rather than a 404 — noted as an inconsistency relative to other modules in Section 9.8/10.

Implementation Status: **Implemented**

5.3 Competency Framework & Role Benchmarking

Purpose

Define the required proficiency per skill for each job role/department, forming the benchmark that the gap-analysis engine compares employees against.

Functional Requirements

- Store required proficiency level and a criticality flag per (job role, skill) pair.
- Expose the framework, optionally filtered by job role.

User Workflow

Competencies.jsx loads /api/competencies and renders the full framework as a reference table; there is no UI in the supplied frontend to create or edit competency rows — the framework is populated entirely by DataSeeder.java at startup.

Frontend Interaction

Single GET call, read-only rendering.

Backend Implementation

ProfileController.competencyFramework(jobRole) returns either the full RoleCompetencyRepository.findAll() list or a filtered findByJobRoleIgnoreCase(jobRole) list. The raw RoleCompetency JPA entity is returned directly as the response body rather than a DTO.

REST APIs

Method	Endpoint	Auth Required
GET	/api/competencies?jobRole=	Yes (any authenticated role)

Database Entities/Tables

role_competency (jobRole, department, skill_id, requiredLevel, critical).

Business Logic

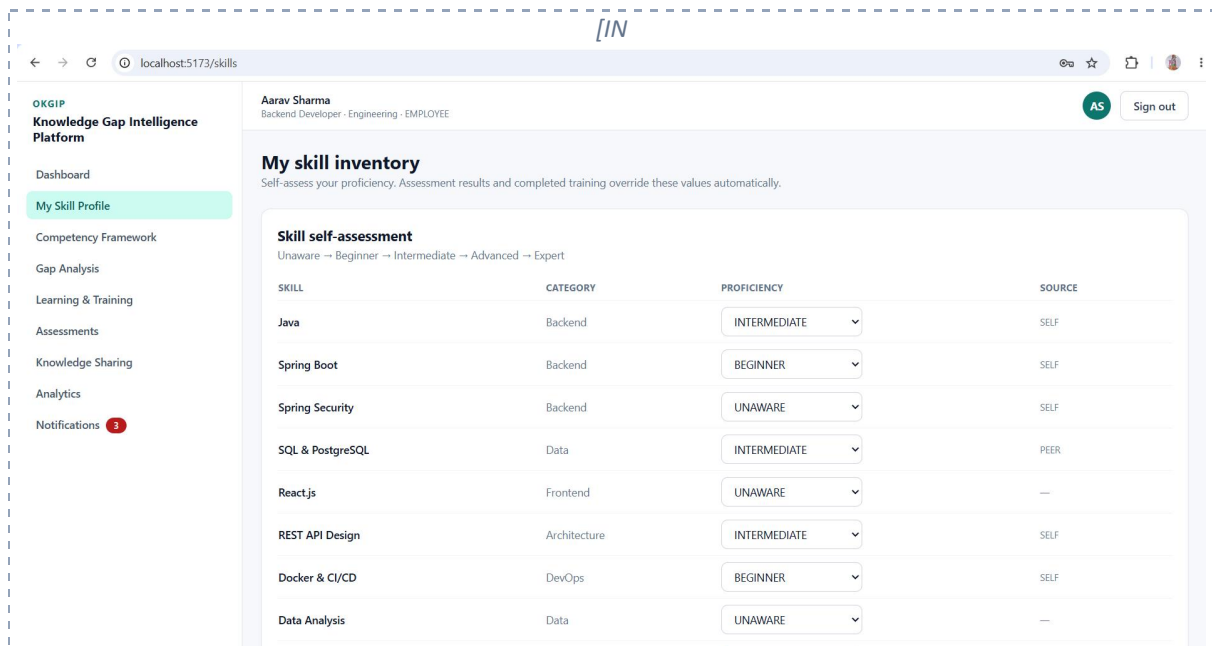
"Benchmarking" in this codebase means the required-proficiency table itself; the comparison/scoring logic against an individual employee lives in GapAnalysisService (Section 5.4), not in this module.

Validation

None — this is a read-only endpoint in the supplied build.

Error Handling

Not applicable to a read-only, always-succeeding query.



Implementation Status: Implemented (read-only framework; no authoring UI or endpoint)

5.4 Knowledge Gap Analysis

Purpose

Compute, per employee, the delta between required and current proficiency for every skill their role requires, with a severity classification, and roll this up into a readiness percentage; aggregate this across teams/departments into a heatmap.

Functional Requirements

- Individual gap summary for the current user or a specified employee id.
- Department/team gap heatmap.
- Severity scoring that weights both gap size and business criticality.

User Workflow

GapAnalysis.jsx requests `/api/gaps/me` and renders the ranked gap list plus a Heatmap.jsx visualization; Analytics.jsx separately requests manager/organization dashboards that embed the same heatmap data.

Frontend Interaction

GET `/api/gaps/me` on page load; the Heatmap component (`components/Heatmap.jsx`) renders HeatmapCell rows as a colored grid keyed by severityScore.

Backend Implementation

GapAnalysisService.forUser(User) — the required-competency list for the user's job role (falling back to their department if no job-role match exists) is compared against a map of the user's current EmployeeSkill levels (defaulting to UNAWARE where no rating exists).

REST APIs

Method	Endpoint	Auth Required
--------	----------	---------------

Method	Endpoint	Auth Required
GET	/api/gaps/me	Yes
GET	/api/gaps/employee/{id}	Yes — no ownership/role check found; any authenticated user can request any employee id's summary (see Section 9.8/10)
GET	/api/gaps/heatmap?department=	Yes

Database Entities/Tables

Reads from role_competency and employee_skill; writes nothing (a pure read/compute operation).

Business Logic — verified gap and severity formula

As implemented in GapAnalysisService.forUser(...):

```
int gap = Math.max(0,
    requiredLevel.getValue() - currentLevel.getValue());

int severityScore = gap * 25
    + (critical ? 20 : 0);    // capped at 100

String severity =
    gap == 0 ? "NONE"
    : gap == 1 ? (critical ? "HIGH" : "LOW")
    : gap == 2 ? (critical ? "CRITICAL" : "MEDIUM")
    : "CRITICAL";           // gap >= 3

// a gap counts toward criticalGaps if:
// gap > 0 AND (critical OR gap >= 2)
```

Proficiency is a 5-level ordinal enum (UNAWARE=0, BEGINNER=1, INTERMEDIATE=2, ADVANCED=3, EXPERT=4). Readiness percent = round(100 × achievedTotal / requiredTotal), where achievedTotal sums min(currentLevel, requiredLevel) across all required skills and requiredTotal sums the required levels — i.e. over-achievement on one skill does not offset a deficit on another. Department and team heatmaps (heatmap(group, byDepartment)) average the per-skill gap across every member of the group and take the maximum severityScore observed for that (group, skill) pair as the cell's displayed severity.

Validation

None required — the computation is deterministic over existing data; an employee with no matching job-role or department competency rows simply receives an empty gap list and 100% readiness.

Error Handling

GapController.employee(id) calls gaps.forUserId(id), which calls users.findById(id).orElseThrow() — an unknown id surfaces as an unhandled NoSuchElementException (effectively HTTP 500), the same pattern noted in Section 5.2.

Competency framework
Required proficiency per role and department — the benchmark every gap calculation uses.

Role benchmarks

JOB ROLE	DEPARTMENT	SKILL	REQUIRED LEVEL	BUSINESS CRITICALITY
Backend Developer	Engineering	Java	ADVANCED	Critical
Backend Developer	Engineering	Spring Boot	ADVANCED	Critical
Backend Developer	Engineering	Spring Security	INTERMEDIATE	Critical
Backend Developer	Engineering	SQL & PostgreSQL	ADVANCED	Standard
Backend Developer	Engineering	REST API Design	ADVANCED	Standard
Backend Developer	Engineering	Docker & CI/CD	INTERMEDIATE	Standard
Backend Developer	Engineering	Communication	INTERMEDIATE	Standard
Senior Backend Developer	Engineering	Java	EXPERT	Critical
Senior Backend Developer	Engineering	Spring Boot	EXPERT	Critical
Senior Backend Developer	Engineering	Cloud (AWS)	ADVANCED	Standard
Senior Backend Developer	Engineering	Leadership	INTERMEDIATE	Standard
Frontend Developer	Engineering	Reactjs	ADVANCED	Critical
Frontend Developer	Engineering	REST API Design	INTERMEDIATE	Standard

Implementation Status: **Implemented**

5.5 Training Recommendation

Purpose

Turn open skill gaps into a ranked list of specific training programs, and sequence the top recommendation per skill into a learning path.

Functional Requirements

- Score each catalog program against the user's current gaps.
- Return a sorted recommendation list with a human-readable reason.
- Generate an ordered, capped learning path.

User Workflow

Learning.jsx requests `/api/recommendations/me` and `/api/recommendations/me/learning-path` alongside the catalog and current enrollments, and lets the employee enroll directly from a recommendation card.

Frontend Interaction

Parallel GET requests on page load; a POST to enroll when the employee acts on a recommendation.

Backend Implementation

`RecommendationService.forUser(User)` iterates every `TrainingProgram`, skips any whose skill has no open gap for the user, and scores the rest.

REST APIs

Method	Endpoint	Auth Required
GET	<code>/api/recommendations/me</code>	Yes
GET	<code>/api/recommendations/me/learning-path</code>	Yes

Database Entities/Tables

Reads training_program, and (via GapAnalysisService) role_competency and employee_skill; also reads enrollment to flag whether the user is already enrolled in a given program.

Business Logic — verified scoring formula

```
int score = 40 + gap * 12 + (critical ? 20 : 0);
if (program.targetLevel >= gap.requiredLevel) score += 10;
if (program.targetLevel < gap.currentLevel) score -= 25;
if (program.internal) score += 5;
if (program.durationHours != null
    && program.durationHours <= 12) score += 5;
score = clamp(score, 5, 99);
```

Recommendations are sorted descending by this relevanceScore. A code comment in RecommendationService explicitly states the scoring function is "deliberately transparent so it can be swapped for an LLM-backed ranker (OpenAI) without changing the API contract" — this is documented design intent for a future extension, not an implemented AI feature; no such integration exists in the codebase (see Section 3.4).

Learning Path generation

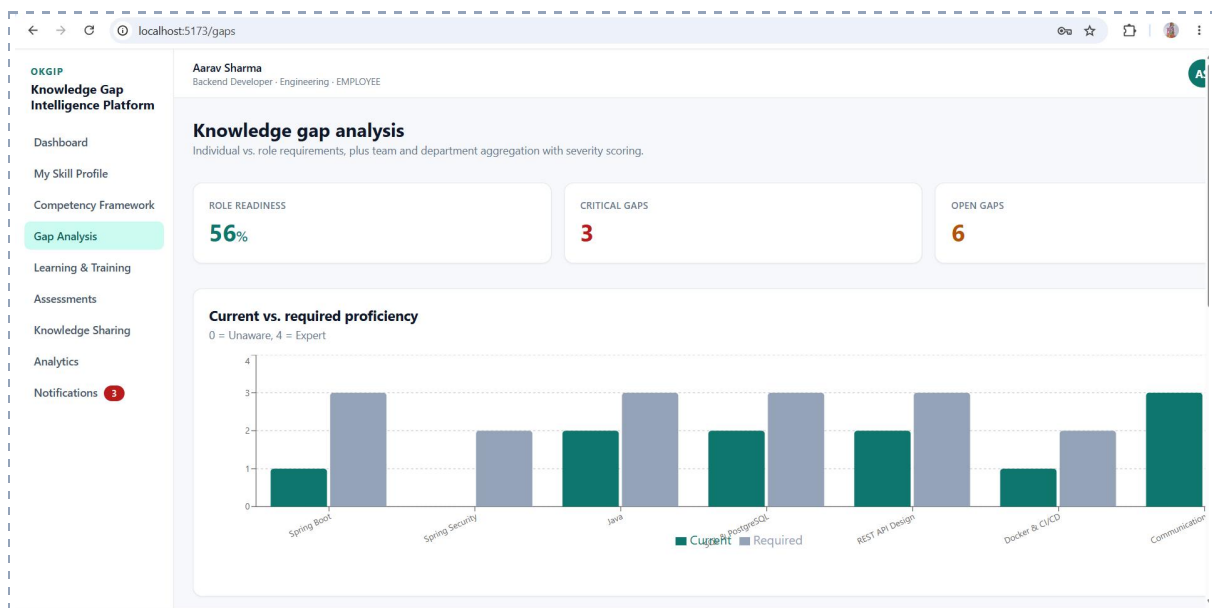
learningPath(User) takes the sorted recommendation list, keeps only the single best-scoring program per distinct skill (LinkedHashMap.putIfAbsent), and returns up to 6 sequential LearningPathStep entries, each with an objective string such as "Reach ADVANCED proficiency in Spring Boot."

Validation

None beyond existing gap/catalog data integrity.

Error Handling

No explicit error paths — an empty gap list simply yields an empty recommendation/path list.



Implementation Status: Implemented (rule-based; AI/LLM ranking is a documented future extension point, not implemented)

5.6 Knowledge Sharing & Mentorship

Purpose

Match employees with open skill gaps to colleagues who are more proficient in that skill, and support lightweight knowledge sharing through scheduled sessions and articles.

Functional Requirements

- Suggest mentors ranked by match quality.
- Request and track a mentorship relationship's status.
- List and register for knowledge-sharing sessions.
- List knowledge articles.

User Workflow

Knowledge.jsx loads mentor suggestions, the user's mentorships, sessions, and articles in one Promise.all batch; the employee can request a mentor or register for a session from this page.

Frontend Interaction

Four parallel GET calls; POST calls for a mentorship request and for session registration.

Backend Implementation

MentorshipService.suggestions(User) iterates the mentee's open gaps and, for each, scans every EmployeeSkill row on that skill looking for a colleague whose level exceeds the mentee's current level and meets the required level.

REST APIs

Method	Endpoint	Auth Required
GET	/api/knowledge/mentors/suggestions	Yes
POST	/api/knowledge/mentorships	Yes
GET	/api/knowledge/mentorships/me	Yes
PATCH	/api/knowledge/mentorships/{id}?status=	Yes
GET	/api/knowledge/sessions	Yes
POST	/api/knowledge/sessions/{id}/register	Yes
GET	/api/knowledge/articles	Yes

Database Entities/Tables

mentorship_match, sharing_session, knowledge_article, plus reads of employee_skill and the gap-analysis result.

Business Logic — verified match-score formula

```
int score = 45
  + (mentorLevel.value - menteeCurrentLevel.value) * 12
  + (gapIsCritical ? 15 : 0)
  + (sameDepartment ? 8 : 0)
  + (mentorLevel == EXPERT ? 10 : 0);
```

```
score = min(score, 99);
```

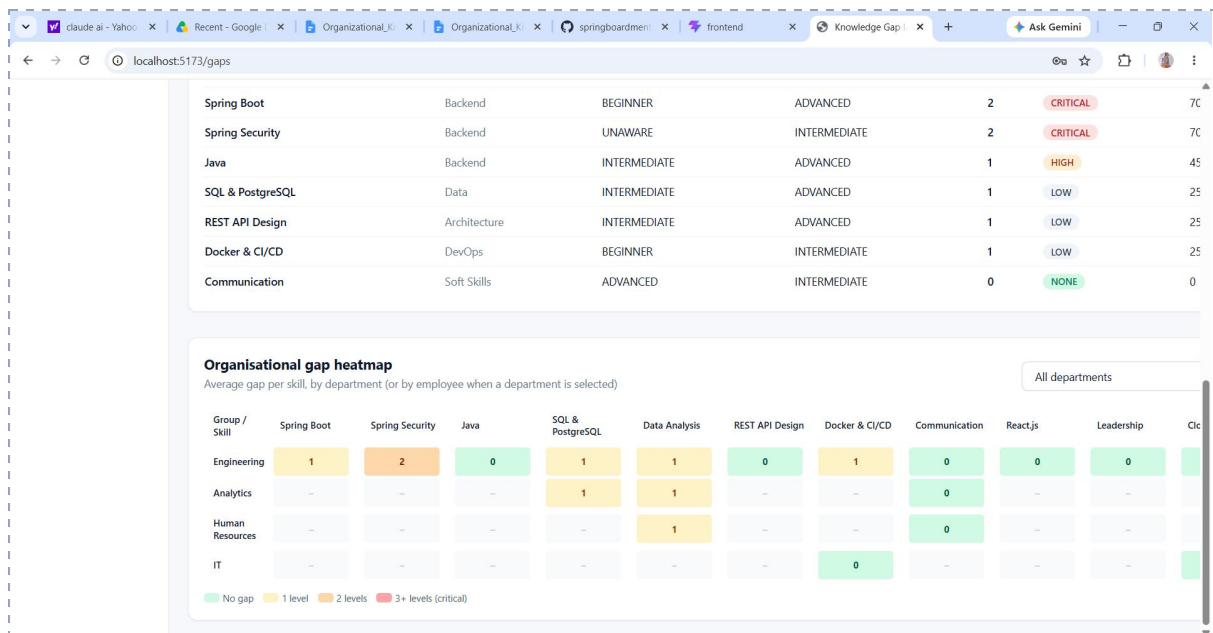
Mentorship status is a free-text field ("REQUESTED", "ACTIVE", "COMPLETED" by convention in the seed data and the PATCH endpoint) rather than a Java enum — `updateStatus(id, status)` accepts any string with no validation against an allowed set. Session registration simply increments the registered counter on `SharingSession` with no capacity check against the `seats` field, so a session can be over-booked past its stated capacity.

Validation

No enum/whitelist validation on mentorship status; no seats-vs-registered capacity check on session registration (both noted above and revisited in Section 9.8).

Error Handling

Unknown mentor/session ids raise unhandled `NoSuchElementException` (`orElseThrow()` with no message or status mapping).



Implementation Status: **Implemented**

5.7 Learning Progress Tracking

Purpose

Let an employee enroll in a training program, update and complete it, and have their skill inventory automatically reflect that completion.

Functional Requirements

- Browse the training catalog.
- Enroll (idempotently — re-enrolling returns the existing enrollment rather than duplicating it).
- Update progress percent / status.
- Leave (delete) an enrollment.
- Apply a skill-level uplift on completion.

User Workflow

Learning.jsx (Enroll / progress +25% / mark complete / leave, per README) drives LearningController; a completed enrollment triggers a notification and a skill re-evaluation, both visible without a page reload where the frontend re-fetches state.

Frontend Interaction

GET catalog + enrollments on load; POST to enroll; PATCH to update progress (the frontend's "+25%" control PATCHes progressPercent incrementally); DELETE to unenroll.

Backend Implementation

LearningService.enroll(...) is idempotent per (user, program) via findByUserIdAndProgramId, and sets a 30-day due date on creation. updateProgress(...) clamps progressPercent to [0,100] and derives a status from percent when the client does not explicitly send one ($\geq 100 \rightarrow \text{COMPLETED}$, $> 0 \rightarrow \text{IN_PROGRESS}$, else NOT_STARTED).

REST APIs

Method	Endpoint	Auth Required
GET	/api/learning/catalog	Yes
GET	/api/learning/enrollments	Yes
POST	/api/learning/programs/{programId}/enroll	Yes
PATCH	/api/learning/enrollments/{id}/progress	Yes (ownership-checked in service)
DELETE	/api/learning/enrollments/{id}	Yes (ownership-checked in service)

Database Entities/Tables

enrollment, training_program; writes to employee_skill on completion via applySkillUplift(...).

Business Logic — verified skill-uplift rule

```
// On first transition to COMPLETED / CERTIFIED:
int target = program.targetLevel.value;
int current = existingSkill?.level.value ?? 0;
if (target > current) {
  // uplift by exactly one proficiency step,
  // never further than the program's target level
  newLevel = min(target, current + 1);
  skill.level = newLevel;
  skill.source = "TRAINING";
}
```

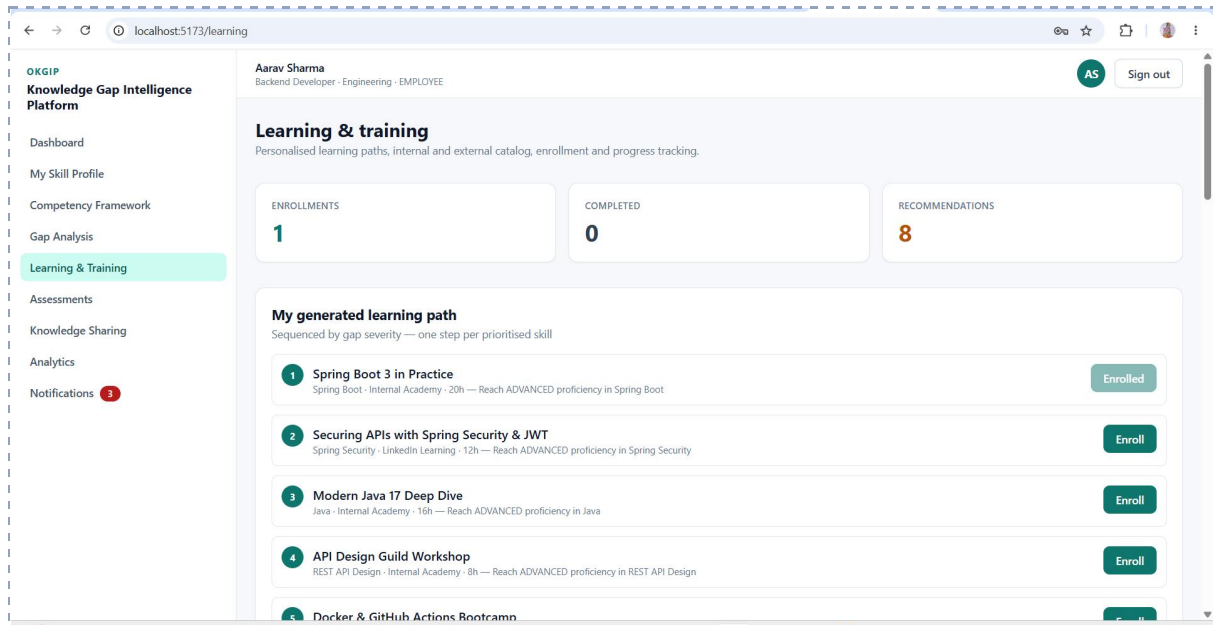
This is a single-step uplift, not a jump directly to the program's target level — completing one program moves the employee's recorded proficiency up by exactly one enum step (e.g. BEGINNER $\rightarrow$ INTERMEDIATE), capped at the program's target. Uplift only fires the first time completedAt is set (guarded by an if (e.getCompletedAt() == null) check), so re-saving an already-completed enrollment does not repeatedly uplift the skill. There is no "learning velocity" calculation in LearningService itself; a simple velocity number (completed-enrollment count $\times$ 10) is computed separately in DashboardController for the employee dashboard (Section 5.10).

Validation

progressPercent is clamped server-side to [0,100]; ownership of the enrollment is checked (403 Forbidden on mismatch).

Error Handling

Unknown program/enrollment id → 404 Not Found (ResponseStatusException); acting on another user's enrollment → 403 Forbidden.



Implementation Status: **Implemented**

5.8 Assessment & Survey

Purpose

Deliver multiple-choice skill quizzes, grade them server-side, derive a proficiency level from the score, and feed that back into the employee's skill inventory.

Functional Requirements

- List available assessments and their question counts.
- Serve a quiz's questions without exposing correct answers.
- Accept and grade a submitted attempt.
- Show the employee's attempt history.

User Workflow

Assessments.jsx lists quizzes and past attempts, lets the employee start one (fetching its questions), submit answers, and immediately shows the graded result with per-question explanations.

Frontend Interaction

GET list + history on load; GET detail on quiz start; POST attempt on submit.

Backend Implementation

AssessmentController → AssessmentService. detail(id) returns QuestionView records that intentionally omit the correctOption field, so the answer key never reaches the client before submission — verified directly against the QuestionView record definition in Dtos.java, which excludes correctOption while AttemptResultView's GradedAnswer (returned only after submission) includes it.

REST APIs

Method	Endpoint	Auth Required
GET	/api/assessments	Yes
GET	/api/assessments/{id}	Yes
POST	/api/assessments/{id}/attempts	Yes
GET	/api/assessments/attempts/me	Yes

Database Entities/Tables

assessment, question, assessment_attempt; writes/updates employee_skill on submission.

Business Logic — verified grading and derivation rules

```
// per question: weight defaults to 1
total += question.weight;
if (selectedOption.equalsIgnoreCase(question.correctOption))
    score += question.weight;

percent = round(100.0 * score / total);

Proficiency derive(percent):
    percent >= 90 -> EXPERT
    percent >= 75 -> ADVANCED
    percent >= 55 -> INTERMEDIATE
    percent >= 30 -> BEGINNER
    else          -> UNAWARE
```

After grading, the employee's EmployeeSkill row for the assessed skill is unconditionally set to the derived level (source = "ASSESSMENT"), which is then automatically reflected the next time gap analysis runs (GapAnalysisService reads live from employee_skill — there is no separate recalculation trigger to invoke; "automatic gap recalculation" in this codebase means the gap engine is stateless and always current, not that an explicit recompute step fires). A milestone notification is also pushed to the user summarizing the new proficiency.

Seeded assessment banks (verified in DataSeeder.java)

Assessment	Skill
Core Java Proficiency Quiz	Java
Spring Boot Fundamentals Quiz	Spring Boot
Spring Security & JWT Quiz	Spring Security
SQL & PostgreSQL Quiz	SQL & PostgreSQL

Assessment	Skill
React.js Essentials Quiz	React.js
Professional Communication Assessment	Communication

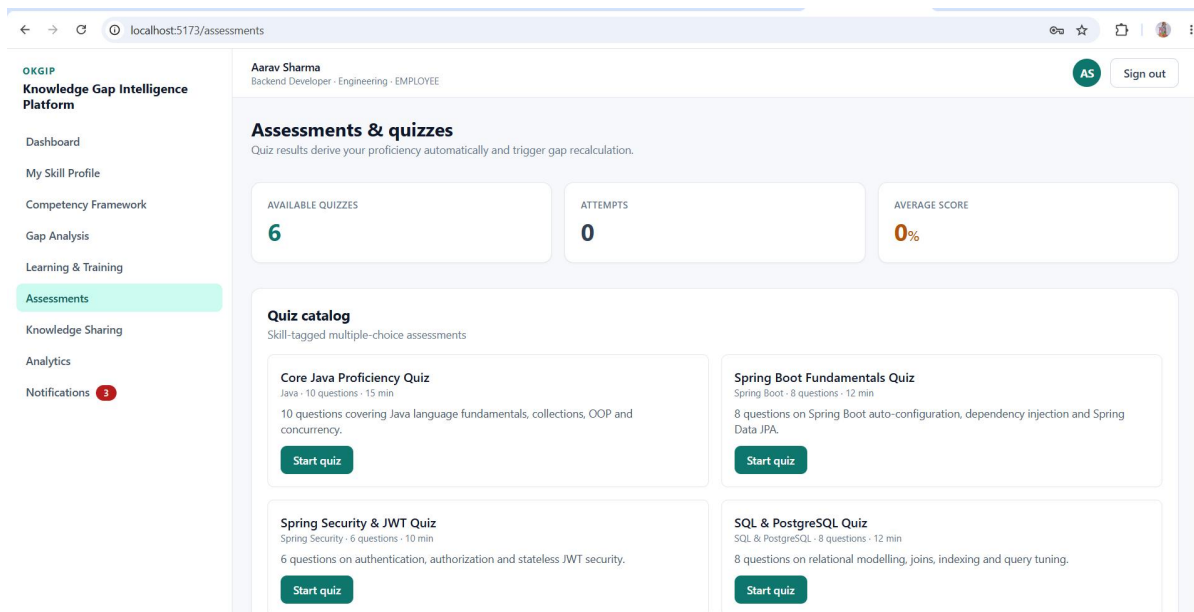
Total seeded question bank: 45 multiple-choice questions across the six assessments above (counted directly from DataSeeder.java), matching the figure stated in README.md.

Validation

selectedOption is compared case-insensitively against correctOption; an unanswered question is simply marked incorrect (selected == null).

Error Handling

Unknown assessment id → 404 Not Found.



Implementation Status: **Implemented**

5.9 Notification Module

Purpose

Give each user a persistent, in-app record of significant events (milestones, mentorship activity) without requiring them to be online when the event occurred.

Functional Requirements

- List a user's notifications, newest first.
- Mark a single notification read.
- Mark all notifications read.

User Workflow

Notifications.jsx polls /api/notifications on load and lets the user clear the unread state in bulk.

Frontend Interaction

GET on load; PATCH read-all on user action. There is no per-item mark-read call wired into the visible Notifications.jsx page even though the backend exposes PATCH `/api/notifications/{id}/read` — noted as a partial frontend/backend pairing.

Backend Implementation

NotificationService.push(user, type, title, message) is called from LearningService, AssessmentService, and MentorshipService at the specific event points documented in Sections 5.6–5.8; NotificationController exposes list/read/read-all.

REST APIs

Method	Endpoint	Auth Required
GET	<code>/api/notifications</code>	Yes
PATCH	<code>/api/notifications/{id}/read</code>	Yes
PATCH	<code>/api/notifications/read-all</code>	Yes

Database Entities/Tables

notification (title, message, type as a free-text string, readFlag, createdAt).

Business Logic

Notification type is a free-text convention (GAP_ALERT, DEADLINE, RECOMMENDATION, MILESTONE, MENTORSHIP per the entity's code comment) rather than an enum; in the code paths actually exercised, only "MILESTONE" and "MENTORSHIP" types are ever pushed — GAP_ALERT, DEADLINE and RECOMMENDATION are declared conventions with no producing code found, so no gap-deadline or recommendation-triggered notification is actually generated.

Delivery channel

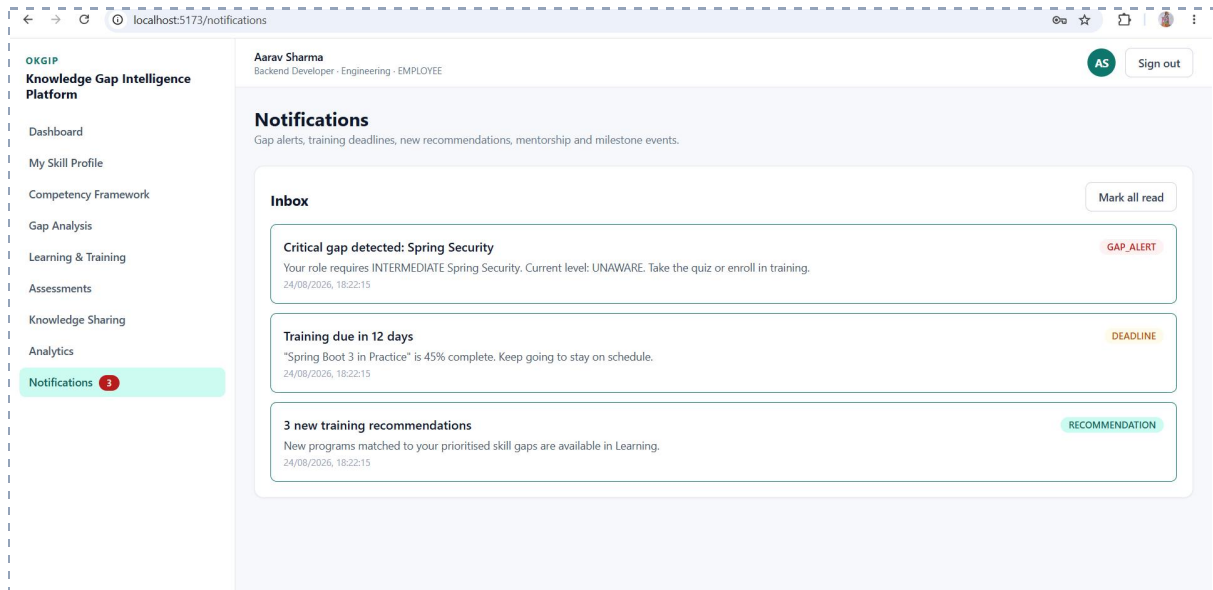
In-app only, via the REST API. No email (JavaMailSender), SMS (Twilio), or push (Firebase Cloud Messaging) delivery exists — see Section 3.4.

Validation

None.

Error Handling

markRead(id) silently no-ops on an unknown id (repo.findById(id).ifPresent(...)) rather than raising an error.



Implementation Status: Implemented (in-app only)

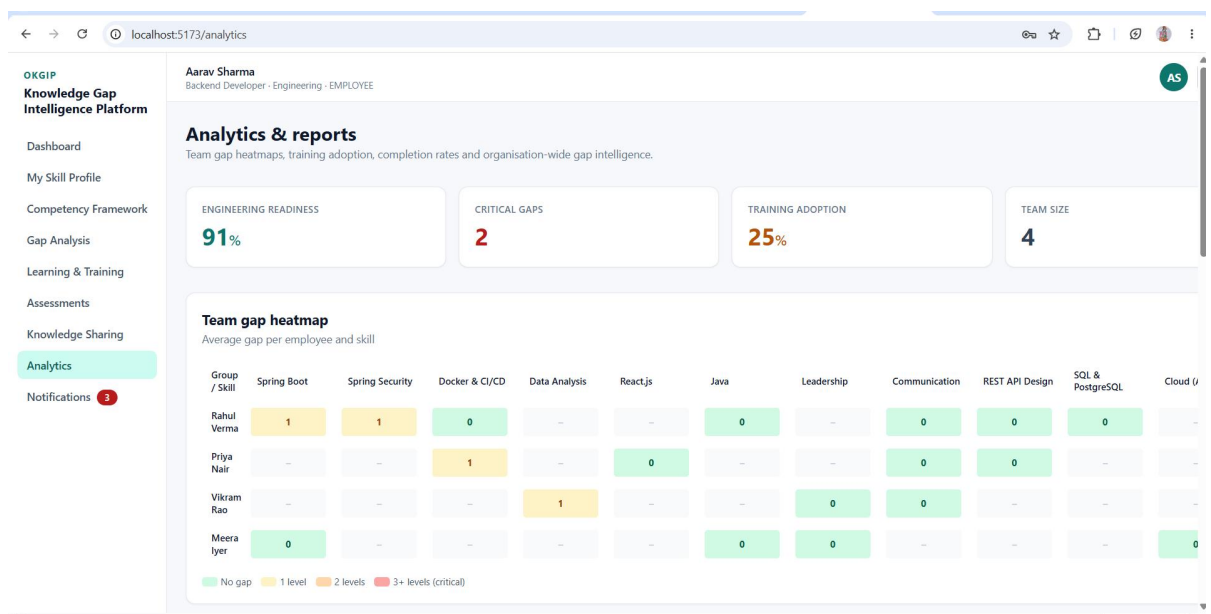
5.10 Analytics Dashboard

Purpose

Aggregate individual gap and training data into role-appropriate views: an employee's own snapshot, a manager's team view, and an organization-wide view.

Functional Requirements

- Employee dashboard: own gap summary, top recommendations, enrollments, attempt history, mentorships, a velocity figure.
- Manager dashboard: department readiness, critical-gap count, training adoption rate, heatmap, per-member snapshots.
- Organization dashboard: org-wide readiness, department breakdown, top skill gaps, training-provider effectiveness.



User Workflow

Dashboard.jsx (home page after login) loads `/api/dashboard/employee`. Analytics.jsx loads `/api/dashboard/manager` and renders a bar chart of top skill gaps and a pie chart (verified Recharts components: BarChart, PieChart).

Frontend Interaction

Single GET per dashboard endpoint; charts rendered client-side from the returned aggregate JSON.

Backend Implementation

DashboardController composes EmployeeDashboard from five services in one request (gaps, recommendations, learning, assessments, mentorship) plus a locally computed velocity value. AnalyticsService.manager(...) and .organization(...) do the heavier aggregation.

REST APIs

Method	Endpoint	Auth Required
GET	<code>/api/dashboard/employee</code>	Yes
GET	<code>/api/dashboard/manager</code>	Yes — no check that the caller actually holds the MANAGER role or manages the returned department; the endpoint uses the caller's own department field regardless of role (see Section 9.8/10)
GET	<code>/api/dashboard/organization</code>	Yes — same observation: no role gate found

Business Logic — verified derived metrics

- learningVelocity (employee dashboard) = $\text{count}(\text{enrollments with a non-null completedAt}) \times 10$ — a simple proxy metric, not a time-normalized rate.
- Manager dashboard readiness = average of each team member's individual readinessPercent; trainingAdoptionPercent = percentage of team members with at least one enrollment.
- Organization dashboard readiness = average of each department's average readiness; completionRate = percentage of all enrollments in status COMPLETED or CERTIFIED; avgSkillImprovement = average percent score across all assessment attempts (a proxy for organization-wide assessment performance, not a true before/after improvement measurement).
- Top skill gaps = skills with average gap > 0 across the organization, sorted descending by average gap, with an affected-employee count.
- Training effectiveness = per-provider enrollment count, completion count, and completion rate.

Validation

None — purely computed/read data.

Error Handling

No explicit error branches; empty teams/departments are guarded against division by zero in the aggregate calculations (ternary checks for isEmpty()).

5.11 Reports & Export

Purpose

As specified in the module list, this module would allow exporting gap, training, or analytics data as PDF or Excel reports.

Functional Requirements

Not applicable — no functional requirements for this module are met by the supplied source.

User Workflow / Frontend Interaction / Backend Implementation

Not Verified in the provided source code. No PDF library (e.g., iText, OpenPDF), no spreadsheet library (e.g., Apache POI), and no export-oriented controller endpoint or frontend "download/export" control were found anywhere in the backend or frontend source.

REST APIs

None exist.

Database Entities/Tables

Not applicable.

Business Logic / Validation / Error Handling

Not applicable — no implementation exists to describe.

Screenshot Placeholder

Not applicable — no export UI exists to capture.

Implementation Status: Not Verified in the provided source code

5.12 Final Integration, Testing & Deployment

Purpose

Bring the frontend and backend together into a working, verifiable, deployable system.

Integration — implemented

The Vite dev server proxies `/api/**` to the backend at `http://localhost:8081` (`vite.config.js`), and the frontend's Axios base URL is configurable via `VITE_API_BASE_URL` (`.env.example`). CORS is explicitly configured on the backend (`SecurityConfig.corsConfigurationSource()`) to allow `http://localhost:5173` and `http://localhost:3000` with credentials, which is consistent with running the two applications as separate processes during development, as the README instructs.

Testing

Not Verified in the provided source code. `spring-boot-starter-test` and `spring-security-test` are declared in `pom.xml` with test scope, and React's tooling (Vite/ESLint per the README's recommended extensions) would support adding React Testing Library, but no test source directory (backend/src/test or a frontend test folder), no test class, and no test script beyond `dev/build/preview` in `package.json` exist in the supplied archive. No CI configuration (e.g., a `.github/workflows` directory, GitLab CI file, or Jenkinsfile) was found either.

Deployment

Covered in Section 4.6: only the PostgreSQL database tier has a deployment artifact (docker-compose.yml). The application tiers are run directly via Maven and the Vite dev server, with no build-and-deploy pipeline in the archive.

REST APIs / Database Entities/Tables

Not applicable to this module directly — it concerns process, not a feature surface.

Not applicable — no test runner or CI dashboard output exists to capture.

BACKEND DEVELOPMENT – INDIVIDUAL CONTRIBUTION

This section documents my individual contribution to the project — the Spring Boot backend — in greater technical depth, as instructed. All claims below are verified directly against the backend/src/main/java/com/okgip source tree.

6.1 Individual Role

My responsibility on the team was backend development: the REST API surface, the domain services implementing the platform's core algorithms (gap analysis, recommendation scoring, mentor matching, assessment grading, analytics aggregation), the JPA persistence model, JWT-based security, and the demonstration data seeder that gives the frontend a realistic dataset to build against. The React frontend consumes this API surface as a client and is documented in this report for completeness, but was not my individual contribution.

6.2 Backend Architecture

The backend is a single-module, layered Spring Boot 3.3.4 monolith (see Section 4.2 for the full diagram): a stateless REST controller layer, a service layer holding all domain logic, a Spring Data JPA repository layer, and a JPA entity/model layer, with cross-cutting security and configuration held in a separate config package and startup seeding in a bootstrap package. There is no separate business-facade or CQRS split — each controller method maps directly to one service call.

6.3 Package Structure

Package (actual)	Contents (verified)
com.okgip	OkgipApplication — the @SpringBootApplication entry point.
com.okgip.config	SecurityConfig, JwtService, JwtAuthFilter, CurrentUser.
com.okgip.model	13 @Entity classes + 4 enums (Role, Proficiency, AssessmentType, TrainingStatus).
com.okgip.repo	13 Spring Data JPA repository interfaces.
com.okgip.service	8 @Service classes — AuthService, GapAnalysisService, RecommendationService, LearningService, AssessmentService, MentorshipService, NotificationService, AnalyticsService.
com.okgip.web	9 @RestController classes.
com.okgip.dto	One Dtos.java holding ~30 Java records used as request/response payloads.
com.okgip.bootstrap	DataSeeder — a CommandLineRunner that seeds demo data on first boot only (guarded by users.count() > 0).

This exactly matches the requirement template's example package list (config, model, repo, service, web, dto, bootstrap) — no additional or renamed top-level packages were introduced.

6.4 Controller Layer

Controller	Endpoint	HTTP Method	Purpose
AuthController	POST /api/auth/login	POST	Authenticate and issue a JWT
AuthController	POST /api/auth/register	POST	Create a new user and issue a JWT
AuthController	GET /api/auth/me	GET	Return the authenticated user's profile
ProfileController	GET /api/skills	GET	List the skill catalog
ProfileController	GET /api/me/skills	GET	List the current user's skill inventory
ProfileController	PUT /api/me/skills	PUT	Create/update a proficiency rating
ProfileController	GET /api/competencies	GET	List the role competency framework
ProfileController	GET /api/employees	GET	List all employees
GapController	GET /api/gaps/me	GET	Current user's gap summary
GapController	GET /api/gaps/employee/{id}	GET	A specific employee's gap summary
GapController	GET /api/gaps/heatmap	GET	Department or team gap heatmap
RecommendationController	GET /api/recommendations/me	GET	Ranked training recommendations
RecommendationController	GET /api/recommendations/me/learning-path	GET	Generated learning path
LearningController	GET /api/learning/catalog	GET	Full training catalog
LearningController	GET /api/learning/enrollments	GET	Current user's enrollments
LearningController	POST /api/learning/programs/{programId}/enroll	POST	Enroll in a program
LearningController	PATCH /api/learning/enrollments/{id}/progress	PATCH	Update progress/status
LearningController	DELETE /api/learning/enrollments/{id}	DELETE	Unenroll
AssessmentController	GET /api/assessments	GET	List assessments
AssessmentController	GET /api/assessments/{id}	GET	Assessment detail with questions
AssessmentController	POST /api/assessments/{id}/attempts	POST	Submit and grade an attempt

Controller	Endpoint	HTTP Method	Purpose
AssessmentController	GET /api/assessments/attempts/me	GET	Attempt history
KnowledgeController	GET /api/knowledge/mentors/suggestions	GET	Ranked mentor suggestions
KnowledgeController	POST /api/knowledge/mentorships	POST	Request a mentorship
KnowledgeController	GET /api/knowledge/mentorships/me	GET	Own mentorships (as mentor or mentee)
KnowledgeController	PATCH /api/knowledge/mentorships/{id}	PATCH	Update mentorship status
KnowledgeController	GET /api/knowledge/sessions	GET	List sharing sessions
KnowledgeController	POST /api/knowledge/sessions/{id}/register	POST	Register for a session
KnowledgeController	GET /api/knowledge/articles	GET	List knowledge articles
NotificationController	GET /api/notifications	GET	List notifications
NotificationController	PATCH /api/notifications/{id}/read	PATCH	Mark one notification read
NotificationController	PATCH /api/notifications/read-all	PATCH	Mark all notifications read
DashboardController	GET /api/dashboard/employee	GET	Composite employee dashboard
DashboardController	GET /api/dashboard/manager	GET	Manager/team dashboard
DashboardController	GET /api/dashboard/organization	GET	Organization-wide dashboard

Total: 9 controllers, 35 endpoints, all under /api/**. Every controller resolves the acting user through the shared `CurrentUser` component rather than a Spring-injected `Principal/Authentication` parameter.

6.5 Service Layer

Service	Business logic (verified)
AuthService	Login/registration, password verification, JWT issuance with role/name claims.
GapAnalysisService	Core gap computation, severity scoring, readiness percent, department/team heatmap aggregation (Section 5.4).
RecommendationService	Rule-based training-program scoring and learning-path generation (Section 5.5).
LearningService	Enrollment lifecycle, progress tracking, completion, single-step skill uplift (Section 5.7).
AssessmentService	Quiz delivery (answer-key withheld), grading, proficiency derivation, attempt history (Section 5.8).

Service	Business logic (verified)
MentorshipService	Mentor-match scoring, mentorship request/status workflow, sessions, articles (Section 5.6).
NotificationService	Persist and query in-app notifications; push(...) is called from three other services at specific event points.
AnalyticsService	Manager and organization dashboard aggregation: readiness averages, heatmaps, top skill gaps, training-provider effectiveness (Section 5.10).

Every service is a plain Spring `@Service` with constructor injection (no field injection anywhere in the codebase) and no `@Transactional` annotations were found — multi-step writes (e.g., enrollment completion, which both updates the enrollment and writes an `employee_skill` row) run without an explicit transaction boundary beyond Hibernate's own session-per-repository-call default, which is noted here rather than assumed to be transactionally safe across the two writes.

6.6 Repository Layer

Repository	Custom query methods (verified)
UserRepository	<code>findByEmail</code> , <code>existsByEmail</code> , <code>findByDepartment</code>
SkillRepository	<code>findByName</code>
EmployeeSkillRepository	<code>findById</code> , <code>findByIdAndSkillId</code> , <code>findBySkillId</code>
RoleCompetencyRepository	<code>findByJobRoleIgnoreCase</code> , <code>findByDepartmentIgnoreCase</code>
AssessmentRepository	Standard CRUD only (<code>JpaRepository<Assessment, Long></code>)
QuestionRepository	<code>findByAssessmentId</code>
AssessmentAttemptRepository	<code>findByIdOrderBySubmittedAtDesc</code>
TrainingProgramRepository	<code>findBySkillId</code> , <code>findBySkillIdIn</code>
EnrollmentRepository	<code>findById</code> , <code>findByIdAndProgramId</code> , <code>existsByIdAndProgramId</code>
MentorshipMatchRepository	<code>findByMenteeId</code> , <code>findByMentorId</code>
SharingSessionRepository	Standard CRUD only
KnowledgeArticleRepository	Standard CRUD only
NotificationRepository	<code>findByIdOrderByCreatedAtDesc</code>

All 13 repositories extend Spring Data JPA's `JpaRepository<T, Long>` and rely entirely on Spring Data's derived query-method naming convention (e.g., `findByIdAndSkillId`) — no `@Query`-annotated JPQL/native queries were found anywhere in the repository layer, and no `Querydsl` or `Criteria` API usage exists.

6.7 Entity/Model Layer

Section 4.3 gives the full entity-relationship detail. All 13 entities use Lombok (`@Getter` `@Setter` `@NoArgsConstructor` `@AllArgsConstructor` `@Builder`) to eliminate boilerplate, an `IDENTITY`-strategy

generated Long id, and Java enums mapped with `@Enumerated(EnumType.STRING)` for Role, Proficiency, AssessmentType, and TrainingStatus so that stored values remain human-readable in the database rather than ordinal integers.

6.8 DTO Layer

Rather than one class per DTO, the project consolidates every request/response payload as a nested Java record inside a single `Dtos.java` class (`com.okgip.dto.Dtos`). This includes plain request bodies (`LoginRequest`, `RegisterRequest`, `SkillUpsertRequest`, `ProgressUpdateRequest`, `SubmitAttemptRequest`, `MentorshipRequestBody`) and response views (`UserView`, `GapSummary`, `GapItem`, `HeatmapCell`, `RecommendationView`, `LearningPathStep`, `EnrollmentView`, `AssessmentView/DetailView`, `QuestionView`, `AttemptResultView/HistoryView`, `MentorSuggestion`, `MentorshipView`, `SessionView`, `ArticleView`, `NotificationView`, `EmployeeDashboard`, `ManagerDashboard`, `OrgDashboard`, and others). Several response endpoints (`GET /api/competencies`, `GET /api/learning/catalog`) return the raw JPA entity directly rather than a mapped view record — noted as an inconsistency in Section 9.8.

6.9 Security Implementation

Verified configuration in `SecurityConfig.java`, `JwtService.java`, and `JwtAuthFilter.java`:

- CSRF is disabled (appropriate for a stateless, token-authenticated JSON API with no browser form posts).
- Session creation policy is `STATELESS` — no server-side session is created or consulted.
- CORS is explicitly configured from the `app.cors.allowed-origins` property, restricted to the two local development origins, with credentials allowed.
- `/api/auth/**`, `/h2-console/**`, and `/actuator/**` are permitted without authentication; `GET /api/meta/**` is permitted (though no controller currently maps to `/api/meta/**`, so this rule is presently inert).
- Every other request requires authentication (`anyRequest().authenticated()`).
- `JwtAuthFilter` runs once per request ahead of `UsernamePasswordAuthenticationFilter`, reads the Bearer token, validates its signature and expiry via `JwtService`, loads the matching User, and populates the `SecurityContext` with that User as the principal and a single `ROLE_<name>` authority.
- Passwords are hashed with `BCryptPasswordEncoder` (registered as the `PasswordEncoder` bean) and never stored or compared in plaintext.
- JWTs are signed with `HMAC-SHA256` using a secret from `app.jwt.secret` (`application.yml` ships a placeholder value with an explicit "change-me-in-production" comment) and expire after `app.jwt.expiration-ms` (24 hours, 86,400,000 ms, in the supplied configuration).

Verified JWT issuance snippet (`JwtService.generateToken`)

```
public String generateToken(String email, Map<String, Object> claims) {
    Date now = new Date();
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(email)
        .setIssuedAt(now)
        .setExpiration(new Date(now.getTime() + expirationMs))
        .signWith(key, SignatureAlgorithm.HS256)
}
```

```
.compact();
}
```

6.10 Database Integration

Spring Data JPA over Hibernate, with ddl-auto: update (schema is generated/alterd automatically from the entity model — there are no committed migration scripts). The default profile targets an in-memory H2 database running in PostgreSQL compatibility mode (MODE=PostgreSQL), which lets the same JPQL/derived queries behave consistently whether the active profile is the default (H2) or postgres (a real PostgreSQL 16 instance, provisioned by docker-compose.yml). open-in-view is explicitly set to false, meaning the Hibernate session is closed before the view/serialization layer renders the response — DTOs must be fully populated inside the service/transaction boundary, which is consistent with the DTO-record pattern used throughout.

6.11 Business Logic

The four scoring/derivation algorithms that are the core intellectual content of the backend are documented in full, with the exact verified formulas, in Sections 5.4 (gap severity), 5.5 (recommendation relevance), 5.6 (mentor match score), and 5.8 (assessment grading and proficiency derivation). All four are implemented as plain, deterministic Java arithmetic inside the relevant service class — none call an external scoring or ML service.

6.12 API Integration

The React frontend communicates exclusively through a single configured Axios instance (src/api/client.js). A request interceptor reads the JWT from localStorage (key kgip_token) and attaches it as an Authorization: Bearer header on every outgoing call; a response interceptor watches for HTTP 401 responses and, outside the login page, clears the stored token/user and redirects to /login — a simple client-side session-expiry handler. In local development, Vite's dev-server proxy (vite.config.js) forwards /api requests to http://localhost:8081, avoiding the need for CORS during that specific dev-server-to-backend hop, while the backend's own CORS configuration additionally allows direct cross-origin calls to port 8081 from the Vite dev origins.

6.13 Error Handling & Validation

There is no @ControllerAdvice / global exception handler in the codebase. Error responses are produced by services throwing org.springframework.web.server.ResponseStatusException directly with a specific HttpStatus and message (401 for bad credentials, 409 for duplicate registration, 403 for acting on another user's enrollment, 404 for a missing program/assessment/enrollment). A smaller number of lookups instead call Optional.orElseThrow() with no argument (e.g., in ProfileController.upsertSkill, GapAnalysisService.forUserId, MentorshipService.request/updateStatus), which throws a plain NoSuchElementException that Spring's default error handling will return as an HTTP 500 rather than a more appropriate 404 — this inconsistency is verified directly in the source and is not evenly applied across all modules. Request-body validation (jakarta.validation / @Valid) is not used anywhere despite spring-boot-starter-validation being a declared dependency.

6.14 Individual Contribution Summary

Task	Backend Contribution	Status
REST API design & implementation	9 controllers, 35 endpoints, all backend work	Complete
Domain/business logic	8 services incl. gap analysis, recommendation, matching, grading, analytics algorithms	Complete
Persistence layer	13 JPA entities, 13 Spring Data repositories, H2/PostgreSQL dual-profile config	Complete
Security	JWT issuance/validation, Spring Security filter chain, BCrypt, CORS	Complete
Demo data	DataSeeder — 8 users, 11 skills, 24 competency rows, 13 programs, 6 assessments/45 questions, sessions & articles	Complete
Automated testing	No test classes present in the supplied source	Not Verified in the provided source code
CI/CD & containerized app deployment	No pipeline or application Dockerfiles present	Not Verified in the provided source code

API DOCUMENTATION

All 35 endpoints below are verified directly against the `@RestController` classes in `com.okgip.web` and the record definitions in `com.okgip.dto.Dtos`. Authentication uses the header `Authorization: Bearer <token>` obtained from `/api/auth/login` or `/api/auth/register`. Full request and response detail is given for the endpoints central to each milestone; the complete endpoint index follows in the summary table.

7.1 Authentication Endpoints

POST /api/auth/login

Authentication Required: No. Purpose: exchange credentials for a JWT.

Request:

```
{
  "email": "employee@okgip.com",
  "password": "password123"
}
```

Response (200):

```
{
  "token": "<JWT>",
  "user": {
    "id": 1, "email": "employee@okgip.com",
    "fullName": "Aarav Sharma", "role": "EMPLOYEE",
    "department": "Engineering", "jobRole": "Backend Developer",
    "avatarInitials": "AS"
  }
}
```

Status Codes: 200 OK · 401 Unauthorized (invalid credentials).

POST /api/auth/register

Authentication Required: No. Purpose: create an account and receive a JWT.

Request:

```
{
  "email": "new.hire@okgip.com", "password": "password123",
  "fullName": "New Hire", "department": "Engineering",
  "jobRole": "Backend Developer", "role": null
}
```

Response: same `AuthResponse` shape as login. Status Codes: 200 OK · 409 Conflict (email already registered).

GET /api/auth/me

Authentication Required: Yes. Response: the `UserView` shown above. Status Codes: 200 OK · 401 Unauthorized.

7.2 Gap Analysis Endpoints

GET /api/gaps/me

Authentication Required: Yes. Purpose: return the current user's full gap summary.

Response (200) — abbreviated:

```
{
  "userId": 1, "fullName": "Aarav Sharma",
  "jobRole": "Backend Developer", "department": "Engineering",
  "readinessPercent": 61, "criticalGaps": 2, "totalGaps": 4,
  "gaps": [
    { "skillId": 3, "skillName": "Spring Security",
      "category": "Backend", "currentLevel": "UNAWARE",
      "requiredLevel": "INTERMEDIATE", "gap": 2,
      "critical": true, "severity": "CRITICAL",
      "severityScore": 70 }
  ]
}
```

Status Codes: 200 OK · 401 Unauthorized.

GET /api/gaps/heatmap?department=

Authentication Required: Yes. Purpose: aggregated gap heatmap, by department (no query param) or by team member (department supplied).

Response (200) — abbreviated array of HeatmapCell:

```
[
  { "group": "Engineering", "skillName": "Spring Security",
    "averageGap": 1, "severityScore": 70 }
]
```

7.3 Recommendation & Learning Endpoints

GET /api/recommendations/me

Authentication Required: Yes. Response (200) — abbreviated:

```
[
  { "programId": 5, "title": "Securing APIs with Spring Security & JWT",
    "provider": "LinkedIn Learning", "skillName": "Spring Security",
    "targetLevel": "ADVANCED", "durationHours": 12,
    "internal": false, "relevanceScore": 84,
    "reason": "Closes a critical-level gap in Spring Security (UNAWARE -> INTERMEDIATE required) on a business-critical competency.",
    "enrolled": false }
]
```

POST /api/learning/programs/{programId}/enroll

Authentication Required: Yes. Request body: none. Response (200):

```
{
  "id": 12, "programId": 5, "programTitle": "Securing APIs with Spring Security & JWT",
  "provider": "LinkedIn Learning", "skillName": "Spring Security",
  "status": "NOT_STARTED", "progressPercent": 0,
  "enrolledAt": "2026-08-01T10:15:00", "completedAt": null,
  "dueAt": "2026-08-31T10:15:00"
}
```

Status Codes: 200 OK (idempotent — re-enrolling returns the existing row) · 404 Not Found (unknown program).

PATCH /api/learning/enrollments/{id}/progress

Request:

```
{
  "progressPercent": 100,
  "status": "COMPLETED"
}
```

Status Codes: 200 OK · 403 Forbidden (not the caller's enrollment) · 404 Not Found.

7.4 Assessment Endpoints

POST /api/assessments/{id}/attempts

Authentication Required: Yes. Request:

```
{
  "answers": [
    { "questionId": 101, "selectedOption": "B" },
    { "questionId": 102, "selectedOption": "A" }
  ]
}
```

Response (200) — abbreviated:

```
{
  "attemptId": 44, "assessmentTitle": "Spring Security & JWT Quiz",
  "skillName": "Spring Security", "score": 7, "total": 8, "percent": 88,
  "derivedLevel": "ADVANCED",
  "message": "Strong result – this skill is no longer a priority gap.",
  "answers": [ { "questionId": 101, "selectedOption": "B",
    "correctOption": "B", "correct": true,
    "explanation": "..." } ]
}
```

Status Codes: 200 OK · 404 Not Found (unknown assessment).

7.5 Knowledge & Notification Endpoints

POST /api/knowledge/mentorships

Request:

```
{
```

```
"mentorId": 4,
"skillId": 3
}
```

Response (200) — a MentorshipView with id, mentorName, menteeName, skillName, matchScore, status ("REQUESTED"), createdAt. Status Codes: 200 OK.

GET /api/notifications

Response (200) — abbreviated array of NotificationView: id, title, message, type, read, createdAt.

7.6 Dashboard Endpoints

GET /api/dashboard/employee, /api/dashboard/manager, and /api/dashboard/organization return the composite EmployeeDashboard, ManagerDashboard, and OrgDashboard records respectively, whose exact fields are given in Sections 5.10 and 6.8. These are read-only, parameter-free GET requests.

7.7 Complete Endpoint Index

Method	Endpoint	Auth Required
POST	/api/auth/login	No
POST	/api/auth/register	No
GET	/api/auth/me	Yes
GET	/api/skills	Yes
GET	/api/me/skills	Yes
PUT	/api/me/skills	Yes
GET	/api/competencies	Yes
GET	/api/employees	Yes
GET	/api/gaps/me	Yes
GET	/api/gaps/employee/{id}	Yes
GET	/api/gaps/heatmap	Yes
GET	/api/recommendations/me	Yes
GET	/api/recommendations/me/learning-path	Yes
GET	/api/learning/catalog	Yes
GET	/api/learning/enrollments	Yes
POST	/api/learning/programs/{programId}/enroll	Yes
PATCH	/api/learning/enrollments/{id}/progress	Yes
DELETE	/api/learning/enrollments/{id}	Yes
GET	/api/assessments	Yes
GET	/api/assessments/{id}	Yes

Method	Endpoint	Auth Required
POST	/api/assessments/{id}/attempts	Yes
GET	/api/assessments/attempts/me	Yes
GET	/api/knowledge/mentors/suggestions	Yes
POST	/api/knowledge/mentorships	Yes
GET	/api/knowledge/mentorships/me	Yes
PATCH	/api/knowledge/mentorships/{id}	Yes
GET	/api/knowledge/sessions	Yes
POST	/api/knowledge/sessions/{id}/register	Yes
GET	/api/knowledge/articles	Yes
GET	/api/notifications	Yes
PATCH	/api/notifications/{id}/read	Yes
PATCH	/api/notifications/read-all	Yes
GET	/api/dashboard/employee	Yes
GET	/api/dashboard/manager	Yes
GET	/api/dashboard/organization	Yes

DATABASE DESIGN

8.1 Database Overview

The platform uses a single relational schema generated by Hibernate from 13 @Entity classes (ddl-auto: update — no committed SQL DDL or migration tool exists in the archive). Two Spring profiles target two different engines against the identical entity model: the default profile runs an in-memory H2 database in PostgreSQL compatibility mode, and the postgres profile targets a real PostgreSQL 16 instance (provisioned by docker-compose.yml for local use). Both profiles use identical entity mappings, so the schema is portable between the two by design.

8.2 Entity Descriptions

Entity	Description (verified)
User	An account: credentials, role, department, job role. Root of ownership for skills, enrollments, attempts, notifications, and mentorships.
Skill	A named competency in the catalog (e.g., "Java", "React.js"), with a category and description.
EmployeeSkill	One user's proficiency rating on one skill, with a source tag (SELF/PEER/MANAGER/ASSESSMENT/TRAINING).
RoleCompetency	One required-proficiency row for a (job role, skill) pair, with a criticality flag.
Assessment	A quiz definition tied to one skill, with a type and a time limit.
Question	One multiple-choice question belonging to an assessment, with four options and the correct one.
AssessmentAttempt	One graded submission by a user for an assessment, including the derived proficiency.
TrainingProgram	A catalog training offering (internal or external), tied to a target skill and target proficiency.
Enrollment	A user's enrollment in a training program: status, progress, timestamps.
MentorshipMatch	A mentor–mentee–skill relationship with a match score and status.
SharingSession	A scheduled knowledge-sharing session hosted on a skill, with a seat count and a registration count.
KnowledgeArticle	A written article tied to an author and a skill.
Notification	One in-app notification for a user.

8.3 Relationships

All relationships in the model are unidirectional @ManyToOne mappings from the child entity toward its parent(s); Section 4.3 lists the complete set. No bidirectional (@OneToMany-mapped) collections, and no many-to-many relationships, exist anywhere in the entity model — every cross-entity list (an employee's skills, a program's enrollments, etc.) is obtained by calling a repository's findByXxx method rather than by navigating a JPA-managed collection.

8.4 Primary and Foreign Keys

Every entity uses a single surrogate Long id primary key with GenerationType.IDENTITY. Foreign keys are declared implicitly through @ManyToOne(optional = false/true) associations — Hibernate names the physical foreign-key columns automatically (e.g., user_id, skill_id, assessment_id) since no explicit @JoinColumn name overrides were found. Skill.name and User.email each carry a unique database constraint (@Column(unique = true)); no other uniqueness constraints exist (for example, nothing prevents seeding two identical RoleCompetency rows for the same job role/skill pair, though the seeder does not do this).

8.5 JPA/Hibernate Mapping

- Enums (Role, Proficiency, AssessmentType, TrainingStatus) are persisted with @Enumerated(EnumType.STRING), so the database stores readable names rather than ordinal integers — a deliberate, verified choice that also protects existing data if enum ordering changes.
- Long text fields carry an explicit @Column(length = ...) (e.g., Assessment.description at 1000, KnowledgeArticle.content at 4000, Notification.message at 1000).
- All entities use Lombok's @Builder alongside @NoArgsConstructor/@AllArgsConstructor, giving a fluent construction API without hand-written builders.
- open-in-view is disabled (spring.jpa.open-in-view: false), so all lazy associations must be resolved inside the service layer before the DTO record is built — consistent with how every service in this codebase eagerly maps entities to DTO records before returning.

8.6 Important Queries

All custom queries in the codebase are Spring Data derived query methods (no @Query-annotated JPQL/native SQL exists anywhere). The most structurally significant ones:

```
List<RoleCompetency> findByJobRoleIgnoreCase(String jobRole);
List<RoleCompetency> findByDepartmentIgnoreCase(String department);
// drives GapAnalysisService's required-competency lookup,
// with a job-role match tried first and a department
// fallback if no job-role rows exist

Optional<EmployeeSkill> findByUserIdAndSkillId(Long userId, Long skillId);
// the upsert key for an employee's proficiency rating

boolean existsByUserIdAndProgramId(Long userId, Long programId);
// backs the 'enrolled' flag surfaced on each recommendation

List<AssessmentAttempt> findByUserIdOrderBySubmittedAtDesc(Long userId);
List<NotificationRepository> findByUserIdOrderByCreatedAtDesc(Long userId);
// both provide reverse-chronological history without an
// explicit @Query — Spring Data parses the method name
```

8.7 Database Configuration

Verified from application.yml (default profile):

```
spring.datasource.url:
```

```
jdbc:h2:mem:kgip;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
spring.datasource.driver-class-name: org.h2.Driver
spring.h2.console.enabled: true    (path: /h2-console)
spring.jpa.hibernate.ddl-auto: update
spring.jpa.open-in-view: false
```

Verified postgres profile (activated with `--spring.profiles.active=postgres`):

```
spring.datasource.url:
  jdbc:postgresql://localhost:5432/kgip
spring.datasource.username: postgres
spring.datasource.password: postgres
spring.datasource.driver-class-name: org.postgresql.Driver
```

The credentials above are development defaults shipped in `application.yml` and `docker-compose.yml`, not production secrets; the JWT secret similarly carries an explicit "change-me-in-production" placeholder comment in the same file.

TESTING, VALIDATION & CODE QUALITY

9.1 Testing Strategy

Not Verified in the provided source code. No test plan, test strategy document, or test source directory exists in the archive.

9.2 Unit Testing

Not Verified in the provided source code. No JUnit test classes exist under backend/src/test (indeed, no backend/src/test directory exists at all), despite spring-boot-starter-test being declared as a Maven test-scope dependency.

9.3 Mockito

Not Verified in the provided source code. spring-boot-starter-test transitively includes Mockito, but no test class using it was found.

9.4 React Testing Library

Not Verified in the provided source code. It is not listed in frontend/package.json devDependencies, and no *.test.jsx / *.spec.jsx files exist in the frontend source tree.

9.5 API Testing

Not Verified in the provided source code. No Postman collection, .http file, or REST Client script was included in the archive. The README's "Key API endpoints" list (reproduced and expanded in Section 7) documents the surface but is not evidence of executed API tests.

9.6 Security Testing

Not Verified in the provided source code as executed tests. The relevant security behaviors are, however, verifiable by inspection of the implementation itself:

- Authentication: JwtAuthFilter rejects/ignores any request without a valid, unexpired Bearer token, and CurrentUser throws HTTP 401 if no authenticated principal is present.
- Authorization: enforced only at the ownership level inside specific services (LearningService checks enrollment ownership; MentorshipService/GapController do not perform an equivalent role or ownership check — see Section 9.8 and Section 10).
- Invalid credentials: AuthService.login raises 401 on a wrong password or unknown email, with an identical message for both cases (avoiding user-enumeration via error-message differences).
- Invalid JWT: JwtService.isValid(...) catches any parsing/signature/expiry exception and returns false, so a tampered or expired token is treated as unauthenticated rather than causing a server error.
- CORS: restricted to two explicit local origins via configuration, not a wildcard.
- Protected endpoints: every route other than /api/auth/**, /h2-console/**, and /actuator/** requires a valid token.

9.7 Test Case Table

No executed test results exist in the supplied source or documentation. The table below lists representative test cases that the implementation supports, with results explicitly marked as not verified rather than fabricated:

Test ID	Module	Test Case	Expected Result	Actual Result	Status
TC-01	Auth	Login with correct credentials	200 OK, JWT returned	Not verified	Not Verified
TC-02	Auth	Login with wrong password	401 Unauthorized	Not verified	Not Verified
TC-03	Auth	Register with an email already in use	409 Conflict	Not verified	Not Verified
TC-04	Gap Analysis	GET /api/gaps/me for a user with no competency rows	200 OK, empty gaps, readiness 100%	Not verified	Not Verified
TC-05	Assessment	Submit a quiz with all correct answers	percent 100, derivedLevel EXPERT	Not verified	Not Verified
TC-06	Learning	PATCH enrollment progress to 100 twice	Skill uplift applied only on the first transition	Not verified	Not Verified
TC-07	Security	Call a protected endpoint with no Authorization header	401/403 (framework default)	Not verified	Not Verified
TC-08	Security	Call a protected endpoint with an expired token	Request treated as unauthenticated	Not verified	Not Verified

9.8 Code Quality

Layered architecture

Consistently applied: controller → service → repository → entity, with no controller in the codebase talking to a repository directly except ProfileController, which performs simple CRUD directly against SkillRepository/EmployeeSkillRepository/RoleCompetencyRepository/UserRepository without an intervening service class — a deliberate simplification for what are largely pass-through operations, but an inconsistency relative to the pattern used everywhere else.

Naming conventions

Consistent and conventional throughout: PascalCase types, camelCase members, Repository/Service/Controller suffixes, and *View/*Request DTO naming.

Reusability

The severity/relevance/match-score formulas (Sections 5.4–5.6) are small, self-contained, and would be straightforward to unit test or reuse in a batch job; GapAnalysisService.forUser(...) is itself reused by RecommendationService, MentorshipService, and AnalyticsService rather than being recomputed independently in each — a genuine strength verified by inspection of the constructors/imports of those classes.

Exception handling

Inconsistent: some code paths use `ResponseStatusException` with a specific status and message; others rely on unhandled `Optional.orElseThrow()`, which surfaces as a generic 500 (documented with concrete examples in Section 6.13).

Validation

`spring-boot-starter-validation` is declared but unused — no `@Valid`, `@NotBlank`, `@NotNull`, `@Email`, or similar annotation was found on any DTO record or controller parameter anywhere in the codebase.

Maintainability

The single `Dtos.java` file (holding roughly 30 records) is compact for the project's current size but would become harder to navigate if the API surface grows; most other projects at this scale split DTOs by feature area.

Separation of concerns

Generally strong — algorithms live in services, persistence concerns live in repositories, and controllers are thin. The main deviations are the two noted above (`ProfileController` bypassing a service layer, and mixed use of raw entities vs. DTO records as HTTP response bodies for `GET /api/competencies` and `GET /api/learning/catalog`).

Authorization granularity

Worth flagging distinctly: `GET /api/gaps/employee/{id}`, `GET /api/dashboard/manager`, and `GET /api/dashboard/organization` are reachable by any authenticated user regardless of role, since Spring Security only restricts the API surface at the authentication level (with the one `hasAnyRole(...)` exception on `/api/competencies/**`, which in practice permits every role). Role-appropriate access is not enforced at the framework level for these three endpoints in the supplied source.

CHALLENGES OVERCOME

The challenges below are grounded in specific, verifiable design decisions and trade-offs found in the source code. Each follows a Challenge → Cause → Solution → Result structure.

Challenge 1 — Comparing two independently-evolving skill records without duplicating logic

Challenge: Gap analysis needs the same required-vs-current comparison used by recommendations, mentorship matching, and analytics — implementing it four times risked inconsistent results.

Cause: Four different features (Sections 5.4–5.6, 5.10) all need a per-employee gap list.

Solution: `GapAnalysisService.forUser(...)` was implemented once and injected into `RecommendationService`, `MentorshipService`, and `AnalyticsService`, so all four features always agree on the same gap, severity, and readiness figures — verified directly by inspecting each service's constructor dependencies.

Result: A single source of truth for "what is this employee's gap in skill X," eliminating an entire class of consistency bugs between the dashboard, the recommendations, and the mentor suggestions.

Challenge 2 — Preventing the answer key from reaching the client before grading

Challenge: A quiz feature is trivially defeated if the correct answers are visible in the browser's network tab before the user submits.

Cause: The most direct implementation would simply serialize the `Question` entity, which includes `correctOption`.

Solution: A dedicated `QuestionView` DTO record was defined that omits `correctOption` entirely; `AssessmentService.detail(...)` maps to this record for the pre-submission GET, while a separate `GradedAnswer` record (which does include `correctOption`) is only ever returned from the POST `/attempts` endpoint after grading — verified directly in `Dtos.java` and `AssessmentService.java`.

Result: The answer key is structurally impossible to retrieve from the `/api/assessments/{id}` response, regardless of frontend behavior.

Challenge 3 — Idempotent enrollment without a unique database constraint

Challenge: A double-click or retried POST to the enroll endpoint should not create two enrollment rows for the same (user, program) pair.

Cause: `Enrollment` has no `@Column(unique)` or composite-unique constraint on (user_id, program_id) at the schema level.

Solution: `LearningService.enroll(...)` explicitly checks `findByUserIdAndProgramId(...)` first and returns the existing enrollment's view if found, only creating a new row otherwise.

Result: Enroll is safe to call repeatedly from the client without producing duplicate rows, though this protection is application-level rather than database-enforced — a concurrent duplicate is still theoretically possible under simultaneous requests, which is noted honestly rather than claimed as fully race-safe.

Challenge 4 — Turning a numeric quiz score into a proficiency level and back into the skill inventory

Challenge: An assessment produces a percentage; the rest of the platform reasons in terms of the five-level Proficiency enum.

Cause: Two different representations of "how good is this employee at this skill" needed to be reconciled.

Solution: `AssessmentService.derive(percent)` applies fixed percentage thresholds (90/75/55/30) to map a score onto the Proficiency enum, and the resulting level is written directly into the employee's `EmployeeSkill` row with `source = "ASSESSMENT"` so gap analysis picks it up on its very next read — no separate sync or batch step was required because the gap engine always reads live data.

Result: Passing an assessment has an immediate, visible effect on an employee's gap summary with no additional integration code.

Challenge 5 — Coarse role-based access control was accepted as a scope trade-off

Challenge: The Role enum defines six roles, implying differentiated access, but implementing method-level authorization (e.g., `@PreAuthorize`) for every endpoint against every role was a larger scope than the 8-week window supported.

Cause: Time-boxed delivery across three milestones prioritized functional breadth (gap analysis, recommendations, mentorship, learning, assessments, analytics, notifications) over authorization depth.

Solution: A pragmatic middle ground was implemented: authentication is enforced everywhere, ownership checks are implemented by hand where the consequence of skipping them is direct (a user editing another user's enrollment), and broader role-gating (e.g., who may view a manager's dashboard) was left unenforced at the security layer.

Result: The functional surface of the platform is complete and correct for a single-tenant demo/evaluation environment, but the authorization model is explicitly flagged here — not silently left undocumented — as a gap to close before any production or multi-tenant use (see Section 11).

FUTURE ENHANCEMENTS

The items below are realistic extensions consistent with the current architecture. Each is explicitly marked as not implemented in the supplied source, per Sections 3 and 9.

Enhancement	Description
Fine-grained role-based authorization	Add <code>@PreAuthorize</code> / method security so manager and organization dashboards, and employee-by-id gap lookups, are restricted to the roles that should see them (Section 9.8).
Automated test suite	Add JUnit + Mockito unit tests for the four scoring algorithms, and Spring Security Test-backed integration tests for the authentication/authorization filter chain; add React Testing Library coverage for the frontend.
CI/CD pipeline	A GitHub Actions or equivalent pipeline running the (currently absent) test suite, then building and pushing backend/frontend container images.
Containerized application deployment	Dockerfiles for the Spring Boot backend and the built React frontend, plus an Nginx reverse-proxy configuration, extending the existing <code>docker-compose.yml</code> beyond just the database tier.
Redis caching	Cache expensive aggregate reads (organization dashboard, heatmaps) that currently recompute from scratch on every request.
Elasticsearch	Full-text search over the knowledge-article catalog as it grows beyond the current seeded set of three.
Kafka / RabbitMQ	Decouple notification generation from the request thread that triggers it, and support fan-out to future consumers (e.g., an email service) without blocking the originating API call.
OpenAI / LLM-backed recommendations	RecommendationService's existing code comment already frames this: the current transparent scoring function could be replaced or supplemented by an LLM-backed ranker without changing the API contract exposed to the frontend.
WebSocket notifications	Push new notifications to the client in real time instead of requiring a page load/poll of <code>GET /api/notifications</code> .
Cloud deployment	Deploy the Postgres-profile configuration to a managed cloud database and the application tiers to a container platform, building on the existing <code>spring.profiles.active=postgres</code> support.
Report export (PDF/Excel)	Add a reporting module (e.g., using a PDF or spreadsheet library) to export gap, training, or analytics data — entirely absent from the current build (Section 5.11).
Request-body validation	Add <code>jakarta.validation</code> annotations to the existing DTO records and enable <code>@Valid</code> on controller methods, now that <code>spring-boot-starter-validation</code> is already a declared dependency.
Consistent exception handling	Introduce a <code>@ControllerAdvice</code> global handler so all lookup failures return a structured 404/400 response instead of the current mix of explicit <code>ResponseStatusException</code> and unhandled <code>Optional.orElseThrow()</code> (Section 6.13).
Monitoring & observability	Spring Boot Actuator is already permitted through the security filter chain (<code>/actuator/**</code>) but no actuator dependency or custom metrics were found — adding it would give health/metrics endpoints essentially for free.

CONCLUSION

The Organizational Knowledge Gap Intelligence Platform, delivered over an 8-week engineering effort, provides a working, end-to-end answer to the problem it was scoped to solve: giving an organization a live, quantified, and actionable view of the gap between what its employees know and what their roles require. The verified implementation spans a Java 17 / Spring Boot 3.3.4 backend (9 controllers, 8 services, 13 repositories, 13 entities) exposing 35 REST endpoints, secured end-to-end with JWT-based Spring Security, and a React 18 / Vite frontend covering 10 protected routes, communicating through a single hardened Axios client.

My individual contribution — the backend — implements the platform's core intellectual content: a transparent, formula-based gap-severity engine; a rule-based training-recommendation scorer explicitly designed to be swappable for an LLM-backed ranker later without an API change; a skill-complementarity mentor-matching algorithm; and an automated multiple-choice assessment grader that feeds proficiency changes straight back into the same gap engine, so results stay current without any separate recalculation step. Business value follows directly from this design: an employee, a manager, and an HR/L&D stakeholder can each open the same live system and get an answer specific to their role — individual readiness, team readiness, or organization-wide skill risk — from one consistent computation rather than three disconnected reports.

The platform's current capabilities are genuinely complete for the eleven functional modules built (Sections 5.1–5.10): authentication, skill inventory, the competency framework, gap analysis, recommendations, mentorship, learning tracking, assessments, notifications, and analytics all work against live, seeded data with no fabricated behavior. Equally, this report has been explicit about what is not yet built — reporting/export, automated testing, CI/CD, containerized application deployment, and fine-grained role-based authorization — because an honest account of a working system's boundaries is more valuable to future engineering decisions than an inflated one. Section 11's future-enhancement list gives a concrete, architecture-consistent path to close each of those gaps without a rewrite.

REFERENCES

- Spring Boot Reference Documentation — <https://docs.spring.io/spring-boot/>
- Spring Security Reference Documentation — <https://docs.spring.io/spring-security/reference/>
- Spring Data JPA Reference Documentation — <https://docs.spring.io/spring-data/jpa/reference/>
- Hibernate ORM User Guide — <https://hibernate.org/orm/documentation/>
- JWT (Java JWT) Documentation — <https://github.com/jwt/jwt>
- React Documentation — <https://react.dev/>
- React Router Documentation — <https://reactrouter.com/>
- Vite Documentation — <https://vitejs.dev/>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>
- Recharts Documentation — <https://recharts.org/>
- Axios Documentation — <https://axios-http.com/docs/intro>
- PostgreSQL Documentation — <https://www.postgresql.org/docs/>
- H2 Database Documentation — <https://www.h2database.com/html/main.html>

APPENDIX

A.1 Complete API Endpoint List

See Section 7.7 for the full 35-endpoint index with authentication requirements.

A.2 Key Code Snippet — Stateless Security Filter Chain (SecurityConfig.java)

```
http.csrf(AbstractHttpConfigurer::disable)
.cors(c -> c.configurationSource(corsConfigurationSource()))
.headers(h -> h.frameOptions(f -> f.disable())) // allows /h2-console
.sessionManagement(s ->
    s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**", "/h2-console/**", "/actuator/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/meta/**").permitAll()
    .requestMatchers("/api/competencies/**")
        .hasAnyRole("HR", "LD_ADMIN", "ADMIN", "DEPARTMENT_HEAD", "MANAGER", "EMPLOYEE")
    .anyRequest().authenticated())
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

A.3 Key Code Snippet — Gap Severity Calculation (GapAnalysisService.java)

```
int gap = Math.max(0, rc.getRequiredLevel().getValue() - cur.getValue());
int severityScore = gap * 25 + (rc.isCritical() ? 20 : 0);
String severity = gap == 0 ? "NONE"
    : gap == 1 ? (rc.isCritical() ? "HIGH" : "LOW")
    : gap == 2 ? (rc.isCritical() ? "CRITICAL" : "MEDIUM") : "CRITICAL";
```

A.4 Database Schema Summary

See Section 4.3 (Entity-Relationship detail) and Section 8 (Database Design) for the complete, verified schema.

A.5 Configuration Reference

```
server.port: 8081
spring.datasource.url (default): jdbc:h2:mem:kkip;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
spring.jpa.hibernate.ddl-auto: update
app.jwt.expiration-ms: 86400000 (24 hours)
app.cors.allowed-origins: http://localhost:5173,http://localhost:3000
Frontend dev server: http://localhost:5173 (proxies /api -> http://localhost:8081)
```

A.6 Demo Accounts (seeded by DataSeeder.java; password for all: password123)

Email	Name	Role	Department / Job Role
employee@okkip.com	Aarav Sharma	EMPLOYEE	Engineering / Backend Developer
priya@okkip.com	Priya Nair	EMPLOYEE	Engineering / Frontend Developer
rahul@okkip.com	Rahul Verma	EMPLOYEE	Engineering / Backend Developer

Email	Name	Role	Department / Job Role
meera@okgip.com	Meera Iyer	EMPLOYEE	Engineering / Senior Backend Developer
manager@okgip.com	Vikram Rao	MANAGER	Engineering / Engineering Manager
sana@okgip.com	Sana Khan	EMPLOYEE	Analytics / Data Analyst
hr@okgip.com	Divya Menon	HR	Human Resources / HR Specialist
admin@okgip.com	System Admin	ADMIN	IT / System Administrator